Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Source Code Management (SCM) DevOps

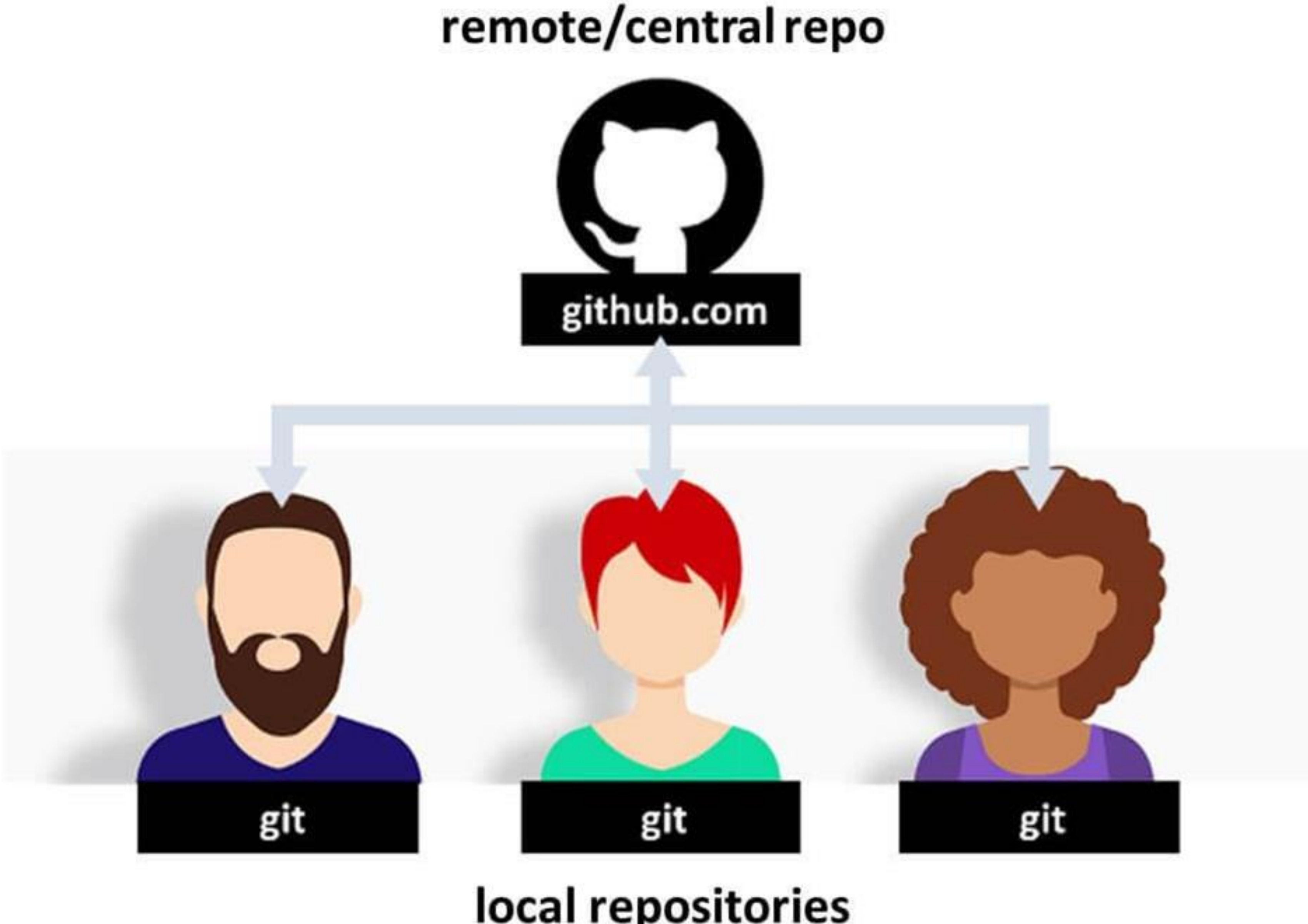
Version Control System (VCS)

Menggunakan Git & GitHub (2023)

Welcome to *the* DevOps Engineering *course*

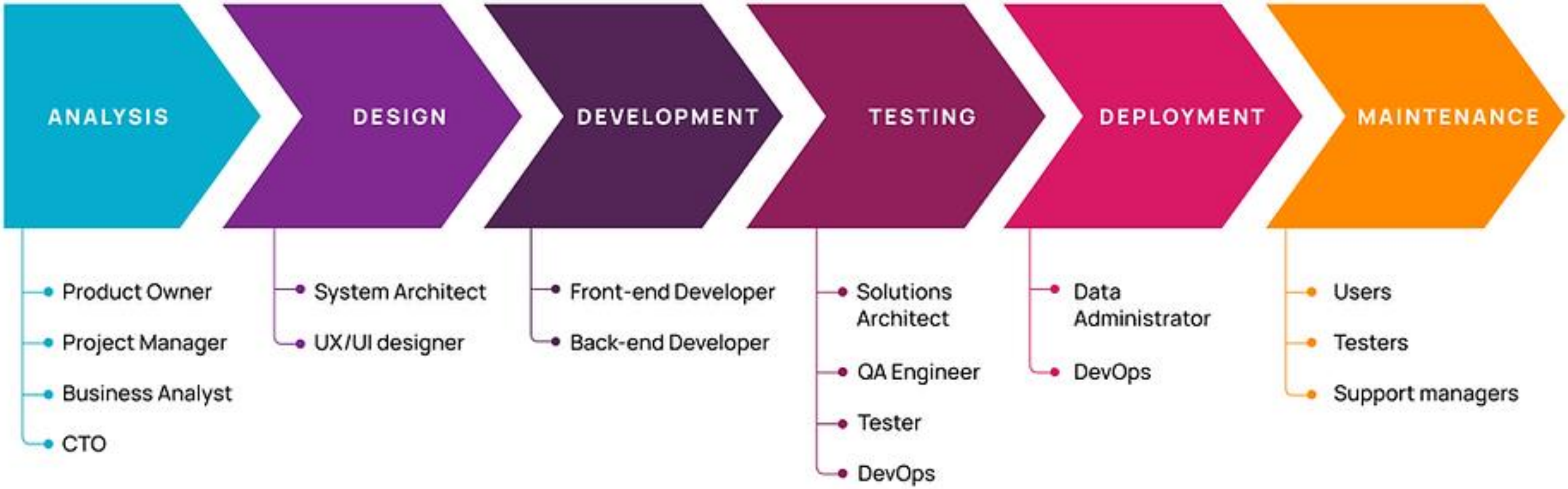
Di class practical sebelumnya kalian sudah melakukan cara manual deploy/edit code di sisi server. Menurut kalian apakah mudah atau sulit?

Kalian bisa bayangkan kalian diminta untuk berkontribusi di dua tim (dev dan ops) tersebut. Pastinya ada banyak tugas yang harus kalian kerjakan.


remote/central repo
github.com
git
git
git
local repositories

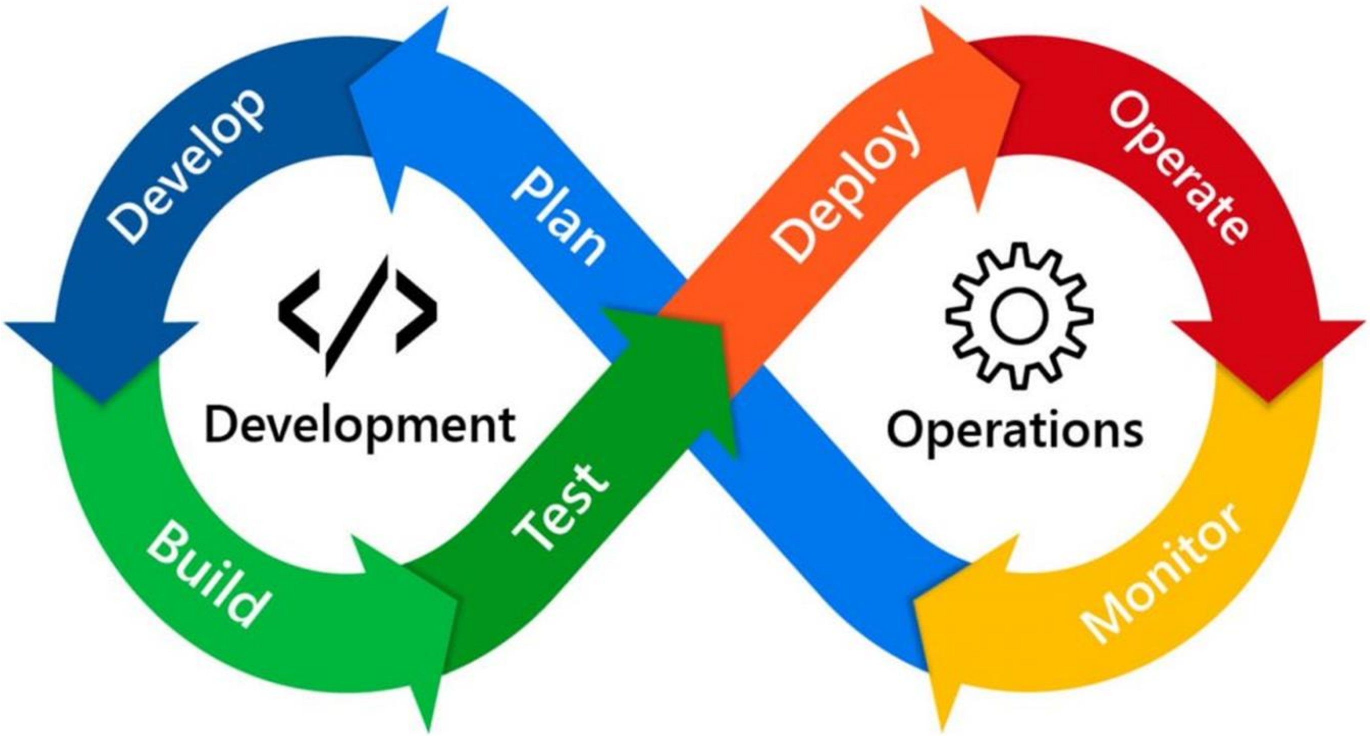
Teamwork is essential in software development, as software development projects are complex processes involving many elements that must work together efficiently.

System Development Life Cycle (SDLC)



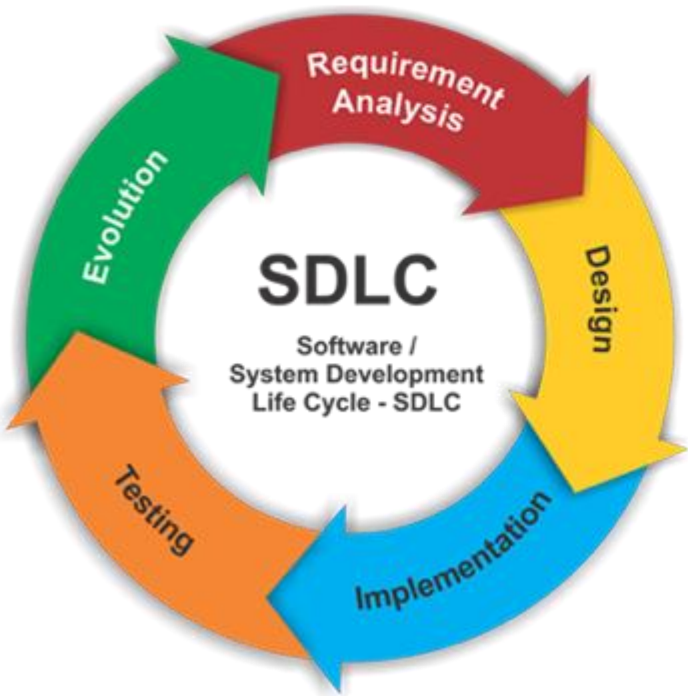
The Software Development Life Cycle (SDLC) is a structured process that enables the production of **<u>high-quality</u>**, **<u>low-cost software</u>**, <u>in the shortest possible production time</u>. The goal of the SDLC is **<u>to produce superior software</u>** that meets and exceeds all customer expectations and demands/requests.

DevOps Methodology



CI/CD stands for Continuous Integration and Continuous Delivery. CI/CD is an effort to **change the software development process from manual to automated**. CI/CD is a software development method that facilitates, accelerates, and improves the quality of the final software released.

(Before) Automation Servers

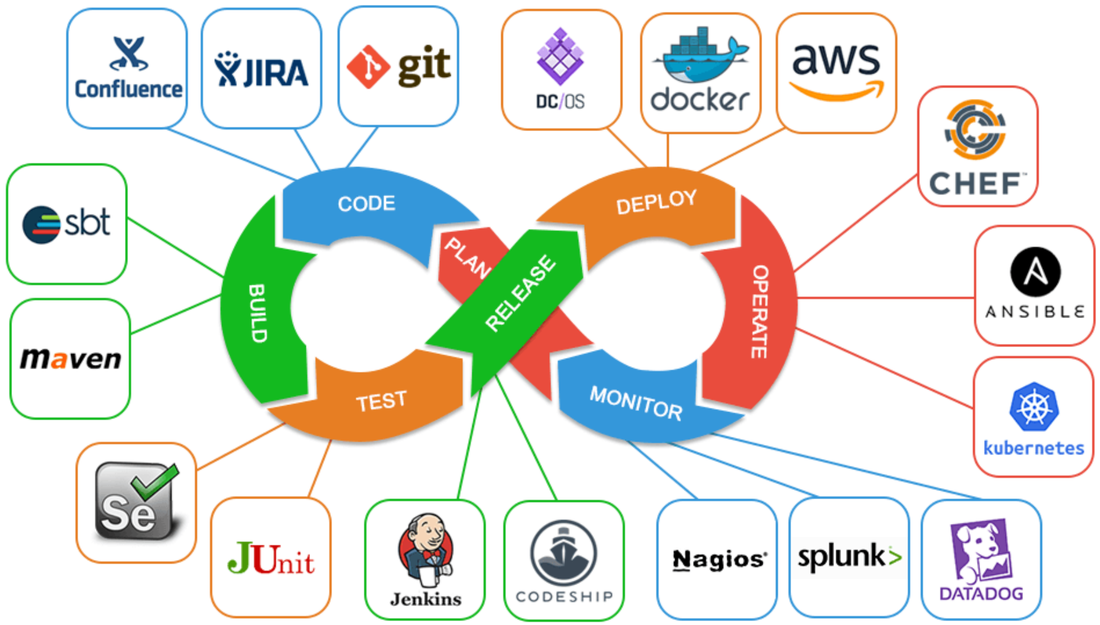


1) Dalam pengembangan software (*perangkat lunak*), koding hanyalah tahapan awal saja.

2) Setelah koding, masih banyak hal yang biasanya dilakukan oleh *software developer* (*perangkat lunak*) sebelum aplikasi yang dibuat sampai ke *production server*.

3) Sebelum adanya Automation Server, biasanya semuanya dilakukan **secara manual**, namun resikonya adalah sering terjadi **kesalahan dan tidak konsisten**.

Contoh menjalankan ***unit test***, download dependency aplikasi, setup database, membuat distribusi aplikasi, dan lain-lain. Hal-hal diatas sangat kompleks jika harus dilakukan secara manual terus menerus.



1) **Automation Server** adalah tool/aplikasi yang bisa kita gunakan untuk **<u>melakukan proses otomatisasi perintah</u>** yang kita instruksikan.

2) Ada banyak sekali keuntungan yang kita dapatkan jika menggunakan Automation Server, dibanding manual dalam tahapan pengembangan software (perangkat lunak).

3) (a) Automation Server **mempercepat proses**, karena kita tidak perlu menunggu di Laptop kita. (b) Automation Server bisa **berjalan konsisten**, di Laptop programmer mungkin perangkat lunak bisa berbeda-beda.

4) Automation Server bisa **berjalan secara otomatis** (hampir sama seperti *scheduler*), bisa memberitahu kita ketika terjadi kesalahan, dan lain-lain.

Version Control (pengendalian versi) adalah komponen penting dalam CI/CD (Continuous Integration/Continuous Deployment) dalam proses DevOps. Ini sangat diperlukan karena membawa sejumlah manfaat yang krusial untuk **manajemen kode**, pengembangan perangkat lunak yang efisien, dan **otomatisasi proses pengiriman**.

Manajemen Konflik

Ketika beberapa pengembang bekerja dalam tim pada proyek yang sama, konflik dalam perubahan kode adalah hal yang umum. Version Control menyediakan mekanisme untuk menangani konflik ini dengan aman dan meresolusi perubahan kode yang bertentangan.

1 – SCM *vs* VCS

Source Code Management (SCM)

- Source code management (SCM) is used to track modifications to a source code repository. SCM tracks a running history of changes to a code base and helps resolve conflicts when merging updates from multiple contributors.

- **SCM is also synonymous with Version control**. SCM is a critical tool to alleviate the organizational strain of growing development costs.

- Source Code Management (SCM) dalam konteks DevOps merujuk pada praktik-praktik dan alat-alat yang digunakan untuk mengelola **source code** dari aplikasi perangkat lunak selama seluruh siklus hidup pengembangan perangkat lunak.

- **SCM merupakan salah satu komponen penting dalam proses DevOps karena memungkinkan tim pengembangan untuk bekerja sama secara efisien, melacak perubahan kode, dan mengotomatisasi aliran kerja pengembangan perangkat lunak**.

Version Control System (VCS)

- Version control system (VCS) adalah sebuah sistem yang **mencatat setiap perubahan terhadap sebuah berkas atau kumpulan berkas (*kode*)** sehingga pada suatu saat developer (*pengguna*) dapat kembali kepada ke salah satu versi dari berkas (*kode*) yang diinginkan.

- VCS sangat mendukung pengembangan *software* yang kolaboratif. Setiap anggota tim development dapat menulis kode programnya masing-masing yang kemudian akan digabungkan menggunakan VCS ini.

- VCS memperbolehkan setiap *developer* untuk

  a) Mengembalikan suatu berkas kode ke keadaan sebelumnya,

  b) Mengembalikan keseluruhan proyek ke keadaan sebelumnya,

  c) Membandingkan perubahan-perubahan berkas kode dari setiap waktu,

  d) Melihat siapa orang yang terakhir mengubah kode yang mungkin menimbulkan konflik, kapan isu muncul, siapa yang memunculkan sebuah isu,

  e) dll.

7 Version Control Systems (VCS)

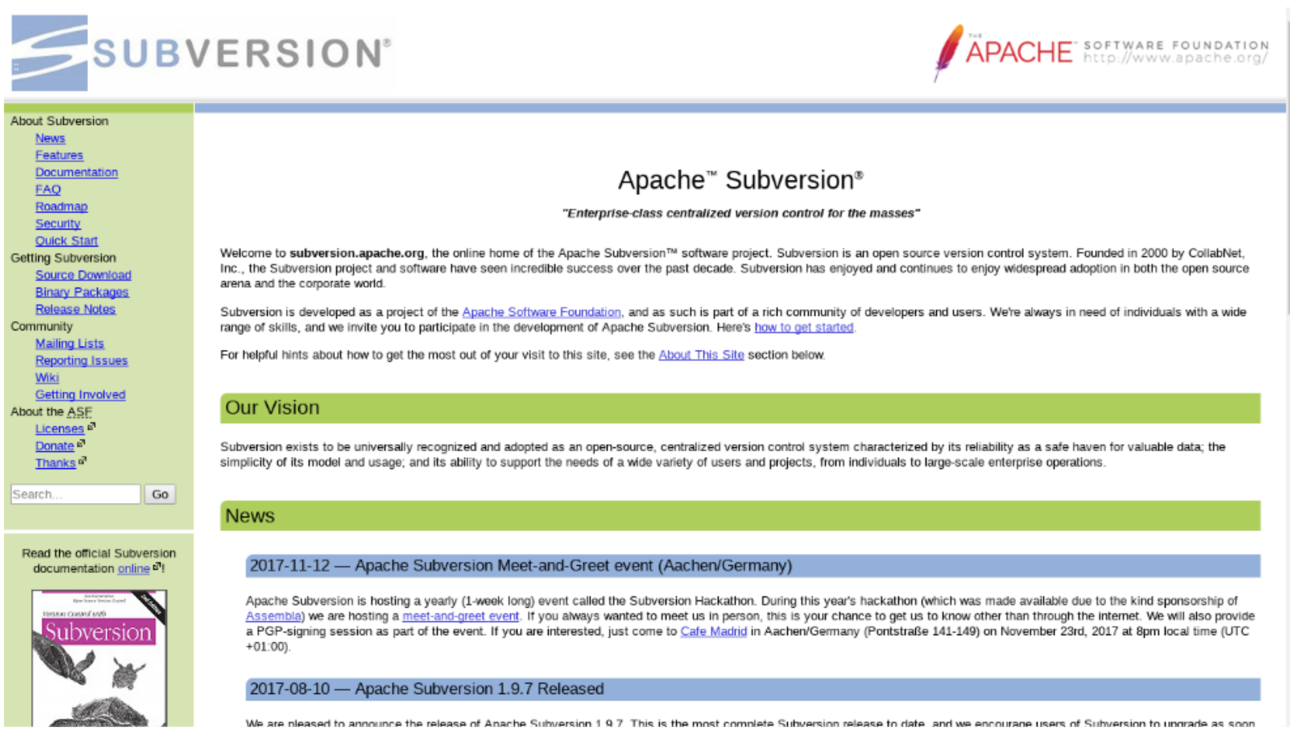
Terdapat banyak Version Control System yang telah dikembangkan para developer. Ada yang berbayar dan ada juga yang free/gratis.

[1] <u>Bazaar</u> merupakan bagian dari GNU Project, Bazaar adalah free software yang disponsori oleh Canonical. Salah satu layanan yang menggunakan Bazaar adalah Launchpad, sebuah tempat dimana aplikasi Ubuntu dikembangkan dan dipantau oleh komunitas. Bazaar dapat digunakan di Windows, Ubuntu, Debian, Red Hat, SUSE, OS X, FreeBSD, Solaris, Gentoo, dan lainnya.

7 Version Control Systems (VCS)

Terdapat banyak Version Control System yang telah dikembangkan para developer. Ada yang berbayar dan ada juga yang free/gratis.

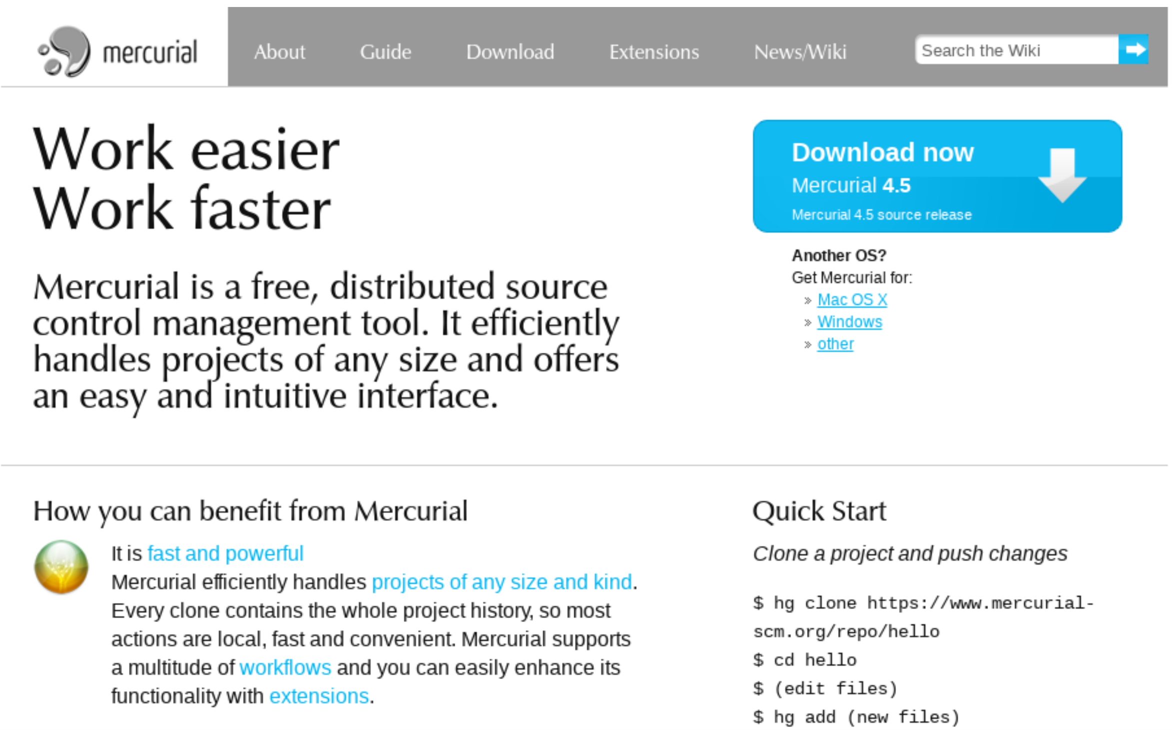
[2] Subversion (SVN) adalah free VCS yang didesain mirip dengan CVS dan lebih sederhana. SVN mendukung atomic commits dari sebuah file dan melakukan versioningterhadap direktori, symbolic links, dan meta-data. SVN merupakan bagian dari Apache Software Foundation. SVN bersifat open source yang didirikan pada tahun 2000 oleh CollabNet Inc. SVN dinikmati luas oleh kalangan komunitas ataupun enterprise. SVN tersedia untuk Linux, OS X, FreeBSD, dan Windows. Saat ini SVN berada di versi 1.9.



7 Version Control Systems (VCS)

Terdapat banyak Version Control System yang telah dikembangkan para developer. Ada yang berbayar dan ada juga yang free/gratis.

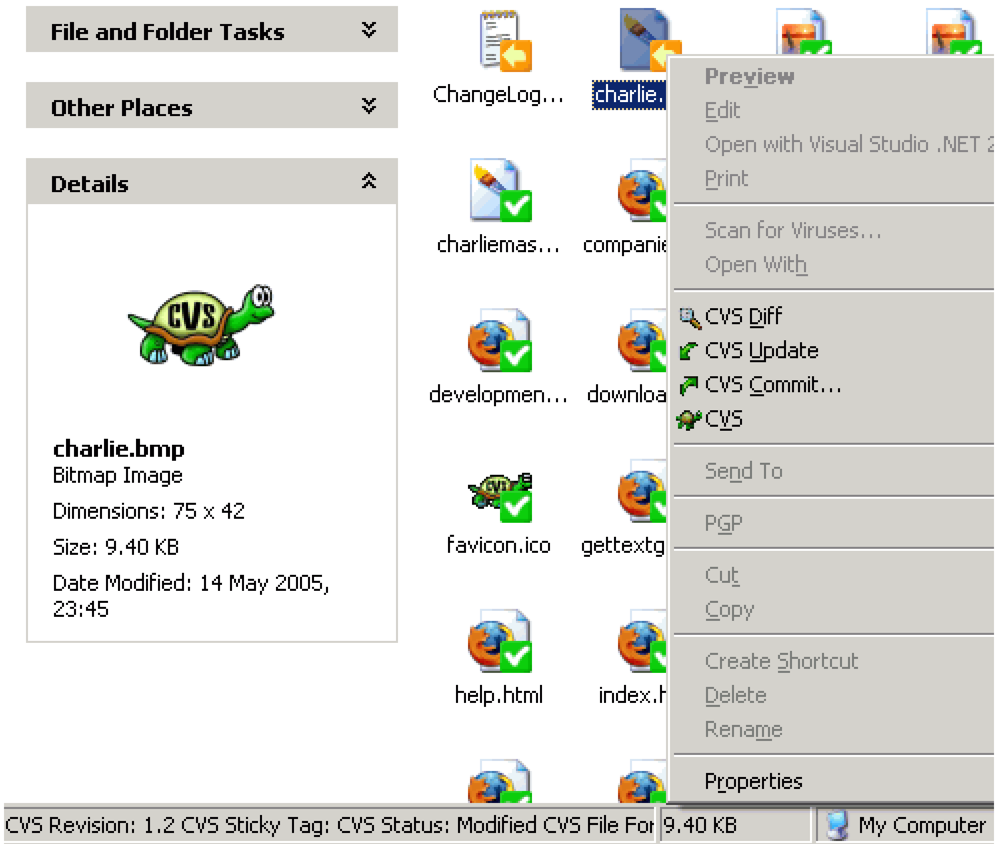
[3] Mercurial merupakan version control system terdistribusi berbasis open source. Mercurial dirancang untuk proyek yang lebih besar, kemungkinan besar berada di luar lingkup designer dan independen Web Developer. Itu tidak berarti tim developer kecil tidak bisa atau tidak boleh menggunakannya. Mercurial memiliki konsep distributed source control management tool. Efisien dalam menangani proyek yang memiliki bermacam ukuran dan jenis. Mercurial mendukung berbagai workflow.



7 Version Control Systems (VCS)

Terdapat banyak Version Control System yang telah dikembangkan para developer. Ada yang berbayar dan ada juga yang free/gratis.

[4] CVS adalah revision control systems yang pertama kali dibuat. pertama kali dirilis pada tahun 1986, dan Google Code masih menjadi tuan rumah posting Usenet yang asli yang mengumumkan CVS.



7 Version Control Systems (VCS)

Terdapat banyak Version Control System yang telah dikembangkan para developer. Ada yang berbayar dan ada juga yang free/gratis.

[5] Revision Control System (RCS) mampu mengelola revisi dari banyak file. RCS mengotomasi penyimpanan, pengambilan, pencatatan, pemeriksaan, dan penggabungan dari sebuah revisi. Berguna untuk teks yang sering direvisi seperti source code, program, dokumentasi, grafik, jurnal, dan surat.

GNU RCS

The Revision Control System (RCS) manages multiple revisions of files. RCS automates the storing, retrieval, logging, identification, and merging of revisions. RCS is useful for text that is revised frequently, including source code, programs, documentation, graphics, papers, and form letters.

RCS was first developed by Walter F. Tichy at Purdue University in the early 1980s -- paper: RCS: A System for Version Control (1991) (troff, PostScript, PDF). See also the Purdue RCS Homepage.

RCS design is an improvement from its predecessor Source Code Control System (SCCS) (see GNU CSSC). The improvements include an easier user interface and improved storage of versions for faster retrieval. RCS improves performance by storing an entire copy of the most recent version and then stores *reverse* differences (called "deltas"). RCS uses GNU Diffutils to find the differences between versions.

Download / News

(FTP mirrors)

Latest release: 5.9.4 (2015-01-22)

- portability fix in `"make check"` for OSX

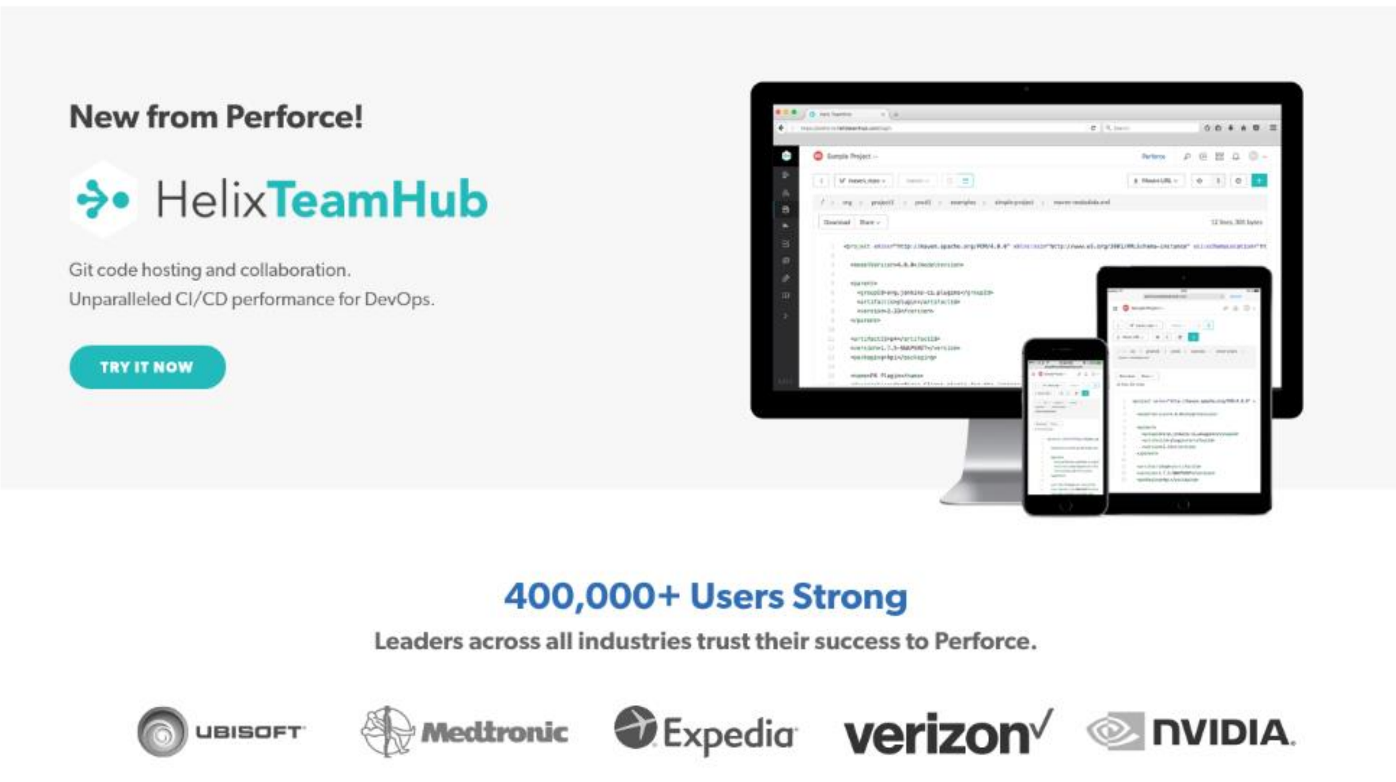
  We now avoid '`head -N`', where N is a number, since that construct is not portable. See: http://lists.gnu.org/archive/html/bug-rcs/2014-11/msg00000.html
- doc improvements
  - index '`rcs -o`' better

    It seems the term `"outdate"` is itself outdated, nowadays (sigh). This command is now indexed under `"deleting"` and `"removing"`, as well as `"outdating"`.

7 Version Control Systems (VCS)

Terdapat banyak Version Control System yang telah dikembangkan para developer. Ada yang berbayar dan ada juga yang free/gratis.

[6] Perforce dipercaya oleh vendor-vendor besar seperti Netflix, Samsung, Salesforce, dan New York Stock Exchange. Perforce memiliki keunggulan seperti massive scalability, hybrid version control, social coding, large binaries, dan unified security. Perforce dapat digunakan di Linux, varian Unix, OS X, dan Windows. Saat ini Perforce berada di versi 2017.3 untuk versi Free.



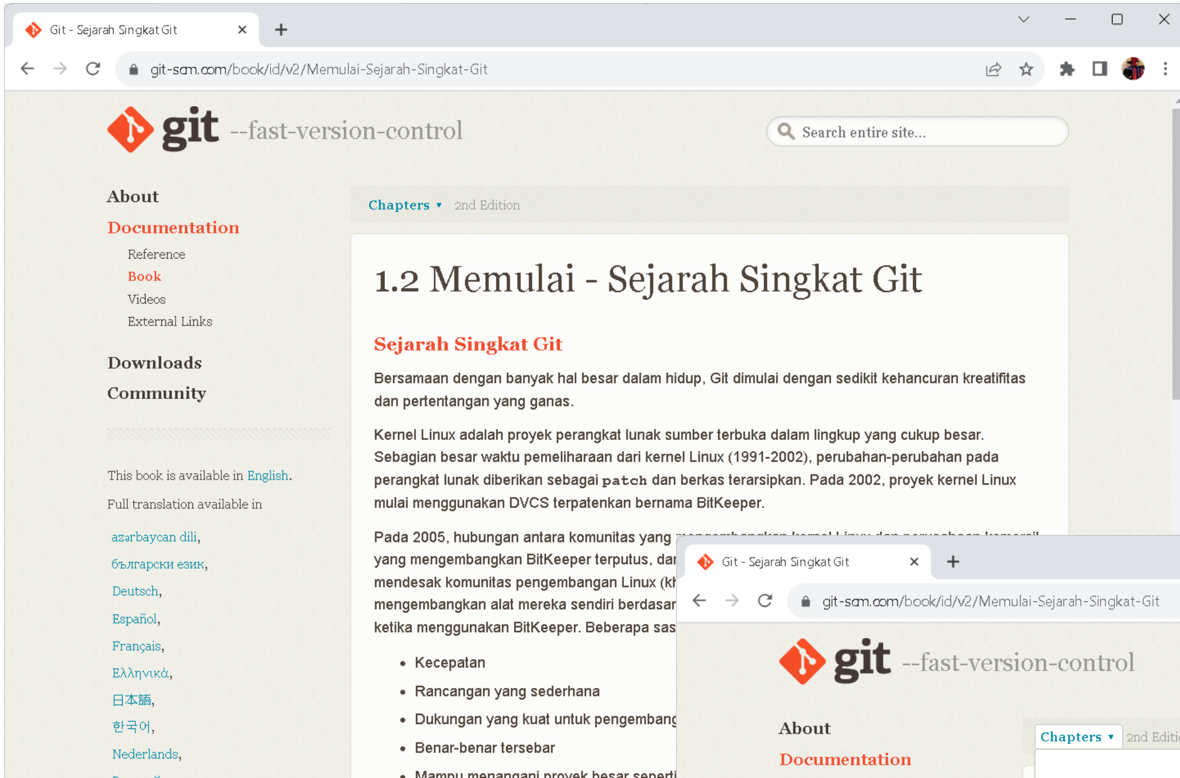
7 Version Control Systems (VCS)

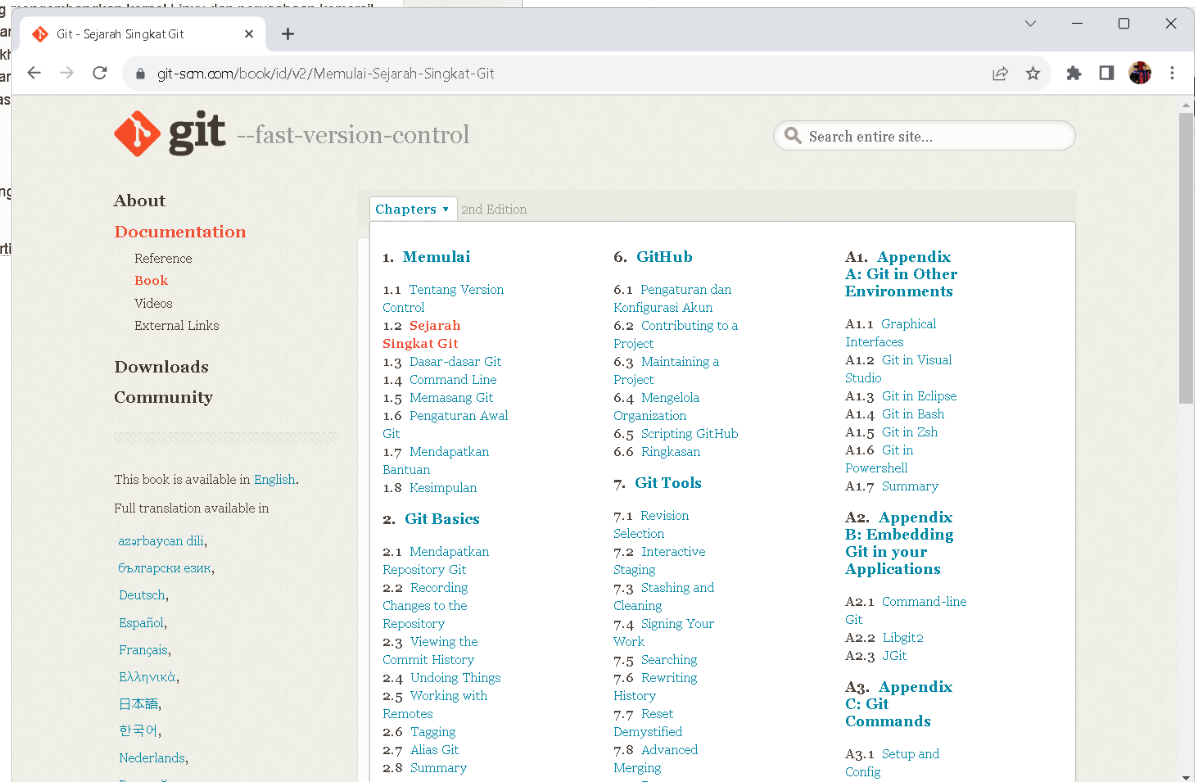
Terdapat banyak Version Control System yang telah dikembangkan para developer. Ada yang berbayar dan ada juga yang free/gratis.

[7] Git merupakan Version Control Sistem yang dikembangkan oleh Linux Torvald ketika mengembangkan kernel Linux pada tahun 2005. Git mengizinkan dan mendorong kita untuk memiliki beberapa cabang lokal yang dapat sepenuhnya independen satu sama lain. Pembuatan, penggabungan, dan penghapusan jalur pengembangan membutuhkan waktu dalam beberapa detik. Git mempunyai keunggulan seperti repository syncing, bekerja secara offline, cheap local branching, staging area yang nyaman, mampu menangani proyek besar seperti Kernel Linux secara efektif dalam hal kecepatan dan ukuran data, mendukung non-linear development, dan multiple workflow.

2 – What *is* Git ???

https://git-scm.com/book/id/v2/Memulai-Sejarah-Singkat-Git





Tutorial Git & Github ubuntu.

What *is* Git?

1) Git adalah **Version Control System (VCS)** tool yang digunakan para developer untuk **mengembangkan software secara bersama-bersama** yang diciptakan oleh Linus Torvalds.

2) Fungsi utama ***git*** yaitu **mengatur versi dari source code program** dengan memberikan tanda baris dan code mana yang ditambah atau diganti.

3) Git memudahkan *programmer* untuk **mengetahui perubahan source code** daripada harus membuat file baru seperti **Program.java**, **ProgramRevisi.java**, ProgramRevisi2.java, ProgramFix.java.

4) Dengan *git* tidak perlu khawatir code yang dikerjakan bentrok, karena setiap developer bisa membuat *branch* sebagai workspace-nya.

5) Selain itu, pada ***git*** bisa memberi komentar pada *source code* yang telah ditambah/diubah, hal ini **mempermudah *developer* lain untuk mengetahui kendala apa yang dialami *developer*** lain.

Distributed Revision Control

1) Pengontrol versi pada Git bertugas **mencatat setiap perubahan pada file proyek** yang dikerjakan oleh banyak orang maupun sendiri.
2) Git dikenal juga dengan **distributed revision control (VCS terdistribusi)**, artinya penyimpanan database Git tidak hanya berada dalam satu tempat saja.
3) **Semua orang yang terlibat dalam pengkodean proyek akan menyimpan database Git**, sehingga akan memudahkan dalam mengelola proyek baik online maupun offline.
4) Dalam Git terdapat merge untuk menyebut aktifitas penggabungan kode.
5) Sedangkan pada VCS (Version Control System) yang terpusat database disimpan dalam satu tempat dan setiap perubahan disimpan ke sana.

- GIT merupakan sebuah Version Control System (VCS) yang digunakan dalam tim pengembangan perangkat lunak untuk bekerja bersama.

- Version Control maksudnya sistem Git akan mencatat setiap perubahan yang terjadi pada source code sehingga memungkinkan untuk **mengambil kembali source code lama jika suatu saat diingin kembali ke versi berapapun** dari aplikasi yang pernah kita tulis.

[IMPORTANT] Git Commands

Git init	untuk membuat *repository* pada file lokal yang nantinya ada folder .git
Git status	untuk mengetahui status dari **repository local**.
Git add	menambahkan file baru pada repository yang dipilih.
Git commit	untuk menyimpan perubahan yang dilakukan, tetapi tidak ada perubahan pada **remote repository**.
Git push	untuk mengirimkan perubahan file setelah di *commit* ke *remote repository*.
Git branch	untuk melihat seluruh *branch* yang ada pada *repository*.
Git checkout	untuk menukar *branch* yang aktif dengan *branch* yang dipilih.
Git merge	untuk menggabungkan *branch* yang aktif dan branch yang dipilih.
Git clone	membuat salinan/copy repository local.

By understanding **Git**, **Git commands**, and **GitHub**, we can more efficiently manage our source code, collaborate with our team, and participate in open-source projects. It can also help us become a more productive and competitive software developer in the IT industry.

Cara Install Git dan Git Bash di Windows:

https://youtu.be/IigslfdR7vI

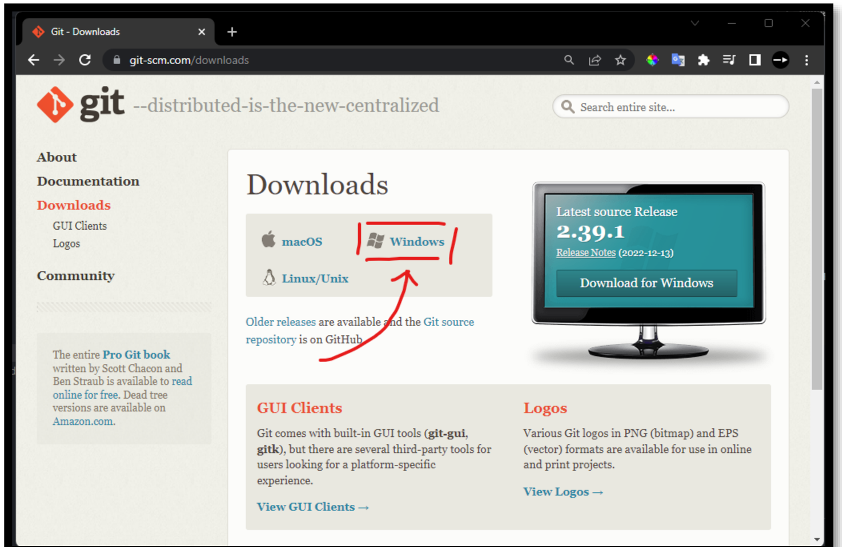
Cara Upload Folder dan File ke Github:

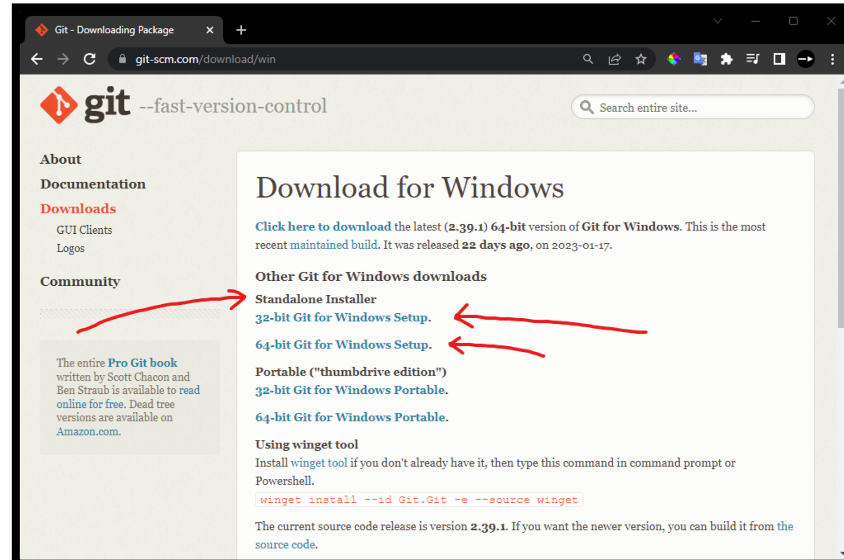
https://youtu.be/WRsK4f9fmY8

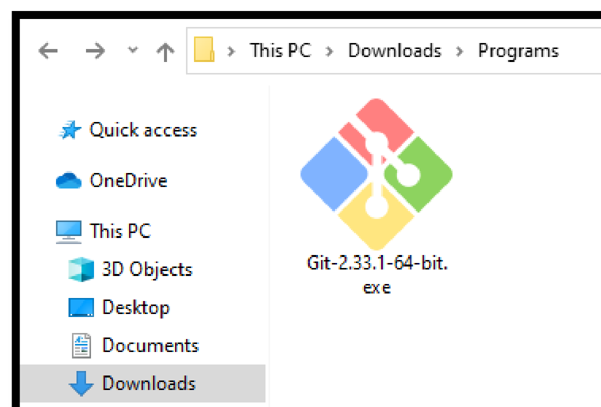
3 – Install Git *on* Windows

Download Installer GIT

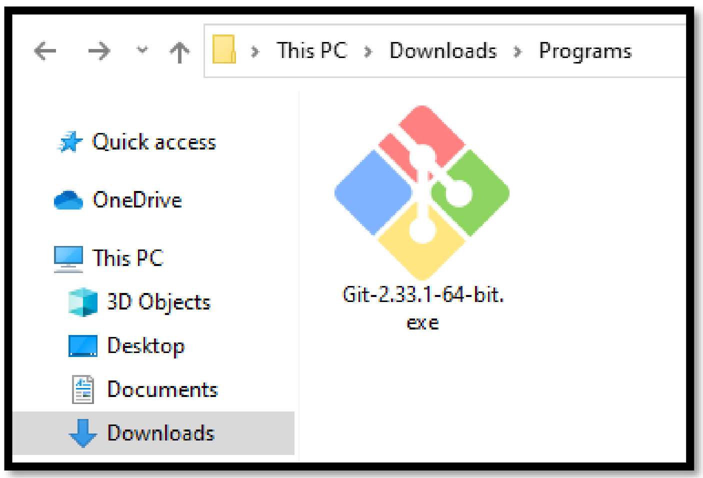
Download Link: https://git-scm.com/downloads

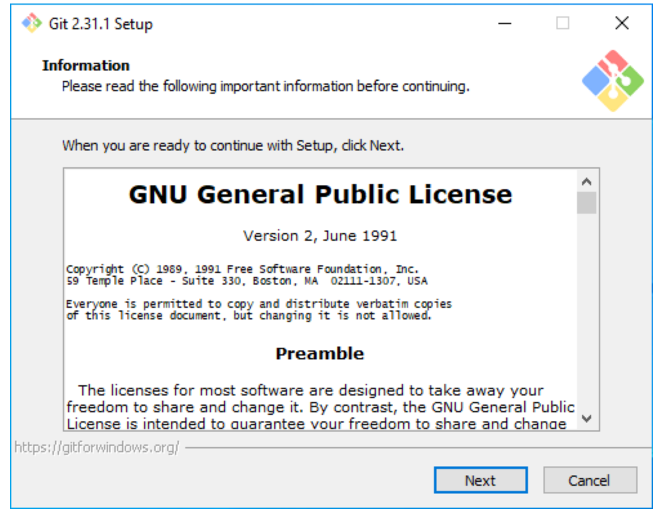


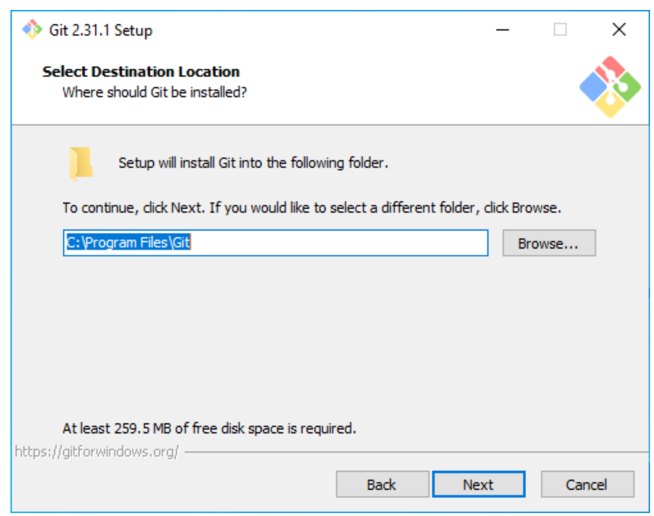




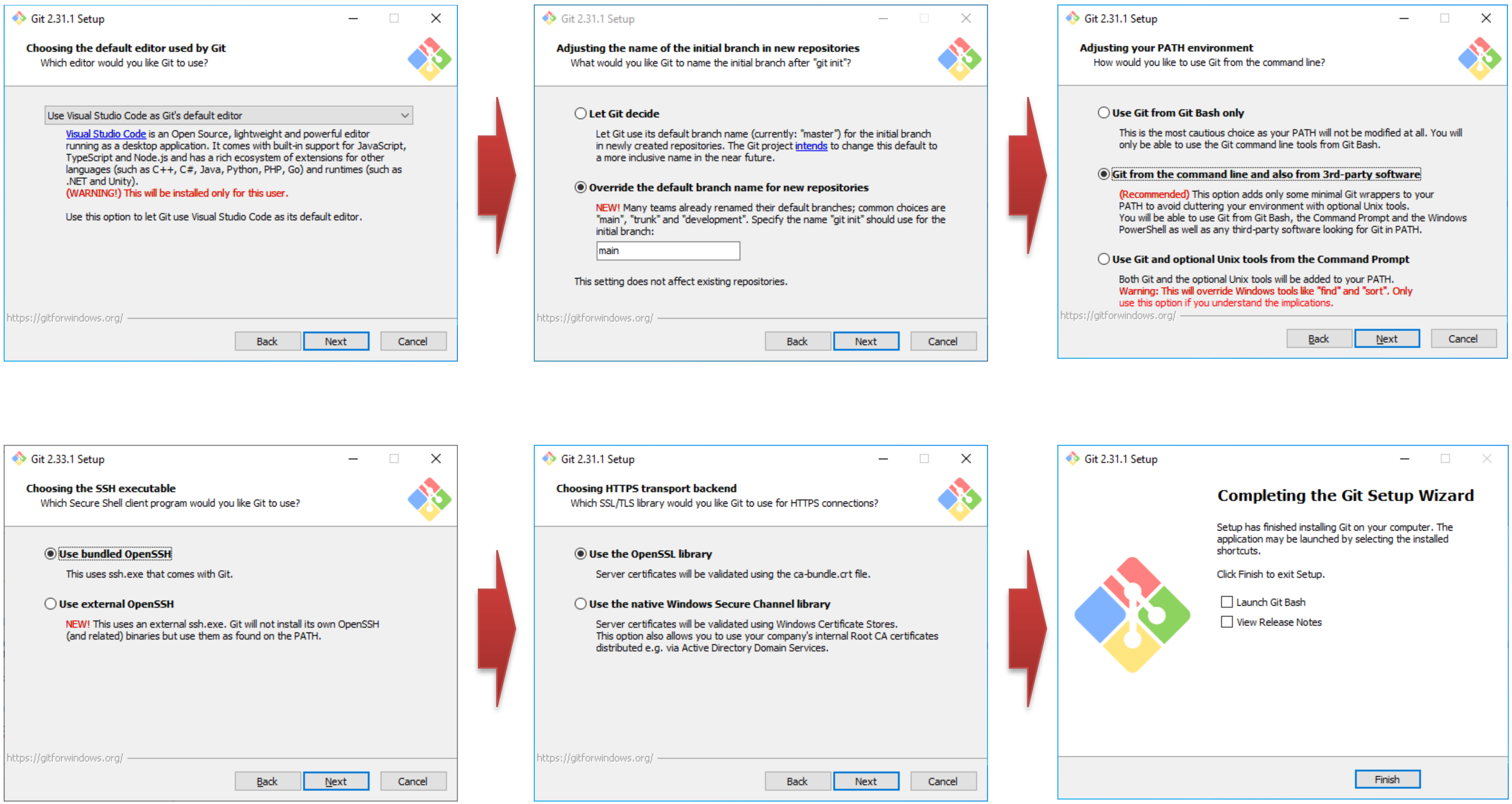
Run Git Installer







Konfigurasi GIT Windows



Tutorial Link: https://aantamim.id/git-windows-install/

How to Run Git on Windows?

Git memiliki **DUA MODE** yang dapat digunakan, git bash scripting shell (*command line*) dan git graphical user interface (GUI).

[1] Git Bash Scripting Shell (Git Bash)

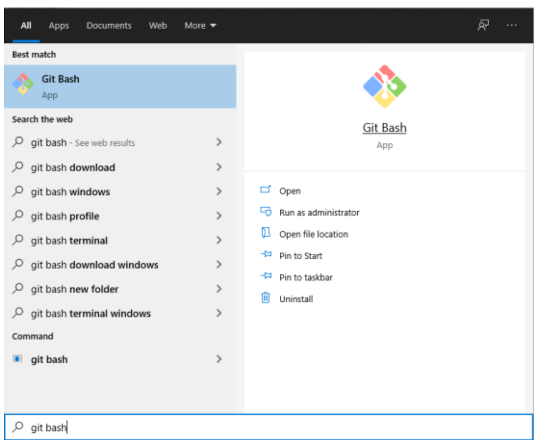
Git Bash adalah sebuah shell atau lingkungan baris perintah yang menjalankan perintah Git. Ini adalah antarmuka berbasis teks di mana kita perlu mengetikkan perintah-perintah Git secara manual. Git Bash tersedia di Windows dan berfungsi seperti lingkungan baris perintah pada sistem operasi Unix.

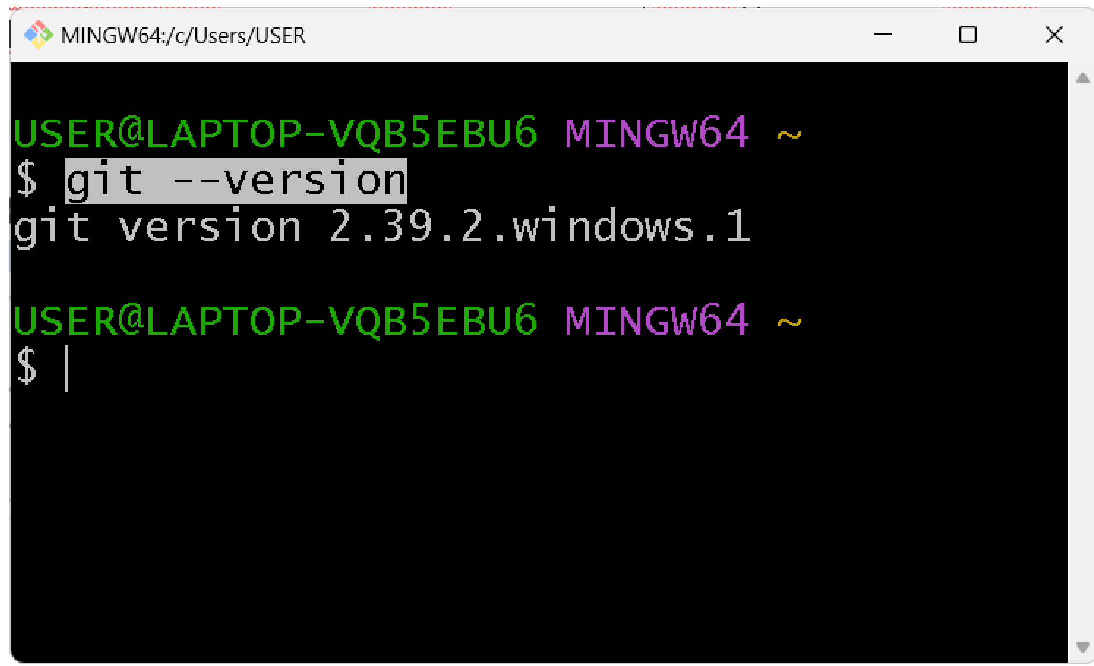
[2] Git GUI (Graphical User Interface)

Git GUI adalah antarmuka/user interface berbasis grafis yang menyediakan visualisasi dan kontrol yang lebih intuitif untuk Git. Ini menawarkan berbagai alat grafis untuk melakukan operasi Git seperti mengelola cabang, menggabungkan perubahan, mengganti komit, dan banyak lagi. Git GUI tersedia dalam berbagai bentuk dan dapat berbeda antara aplikasi dan platform yang berbeda.

Run/Open "*Git Bash Shell*"

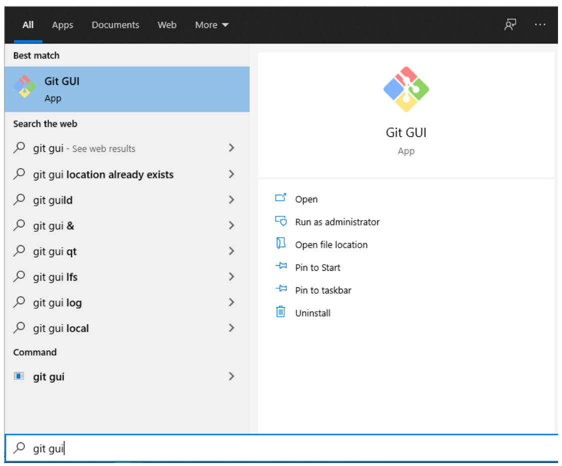
Untuk menjalankan Git Bash buka Start Menu, ketik git bash dan tekan Enter (atau klik dobel pada icon aplikasi).

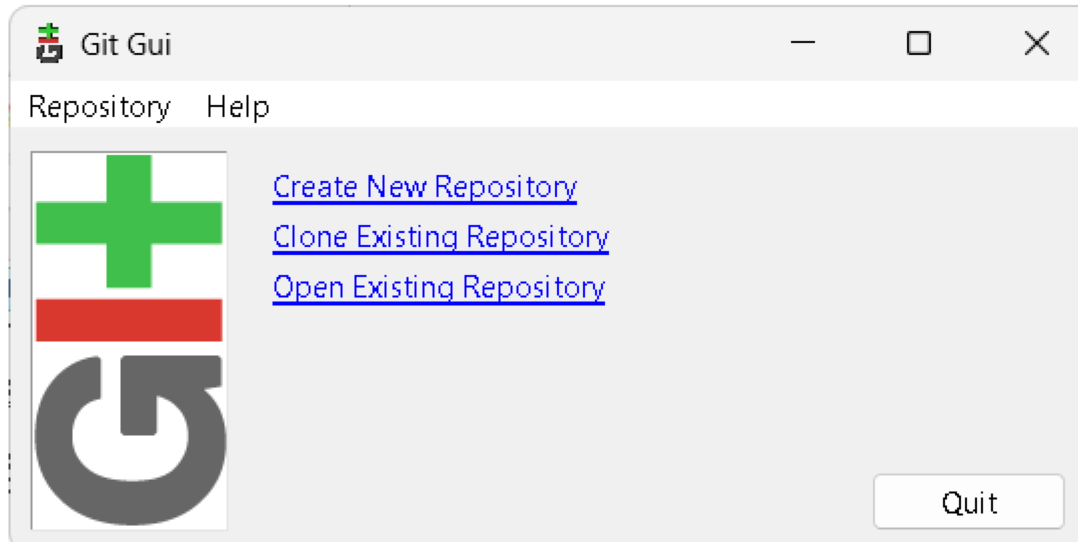




Run/Open “*Git GUI*”

Untuk menjalankan Git GUI buka Start Menu, ketik git gui dan tekan Enter (atau klik dobel pada icon aplikasi).



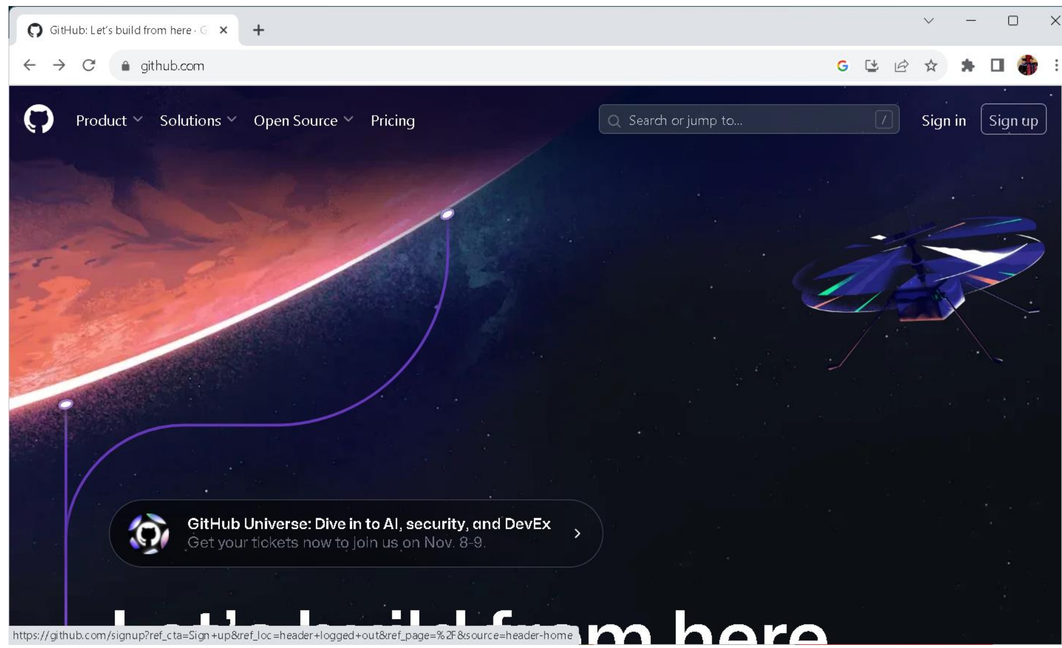


4 – How *to* Create Github Account?

Github Link:

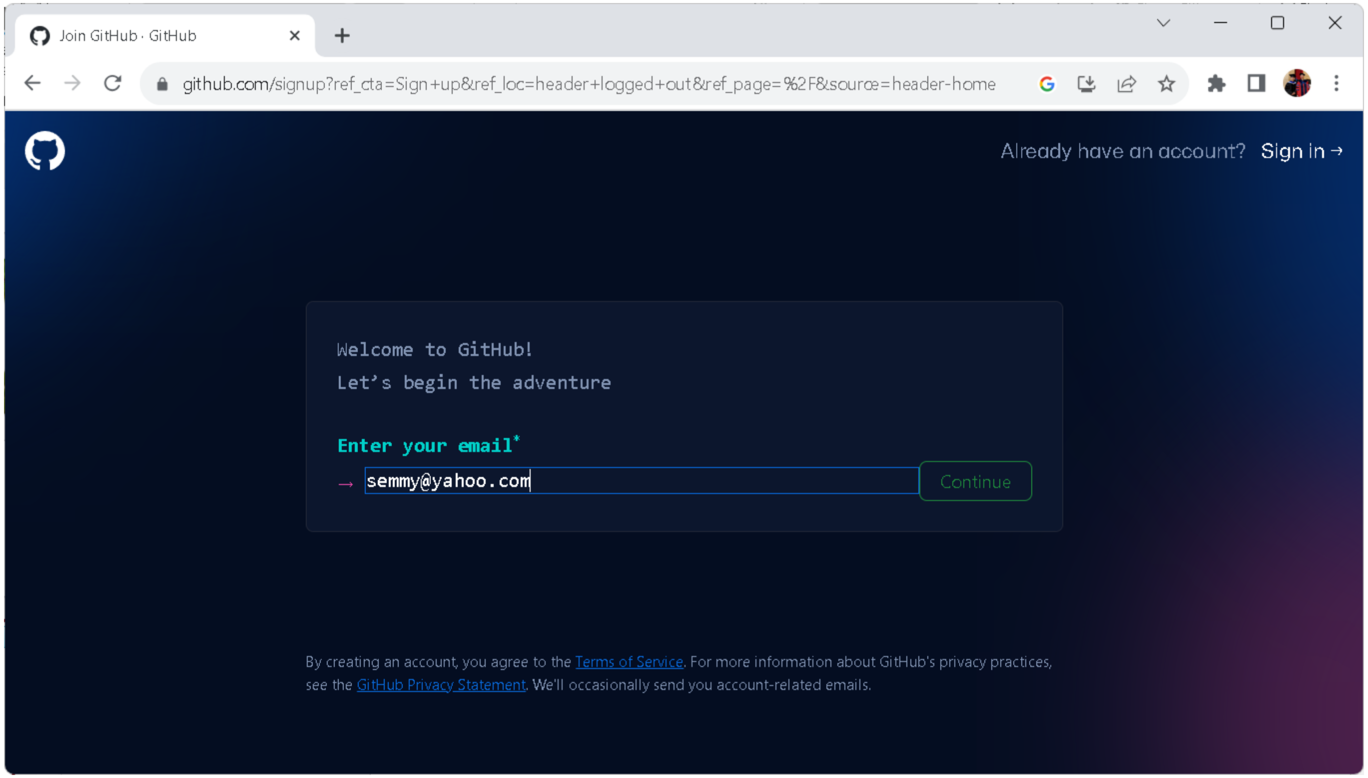
https://github.com/

Open Link Github

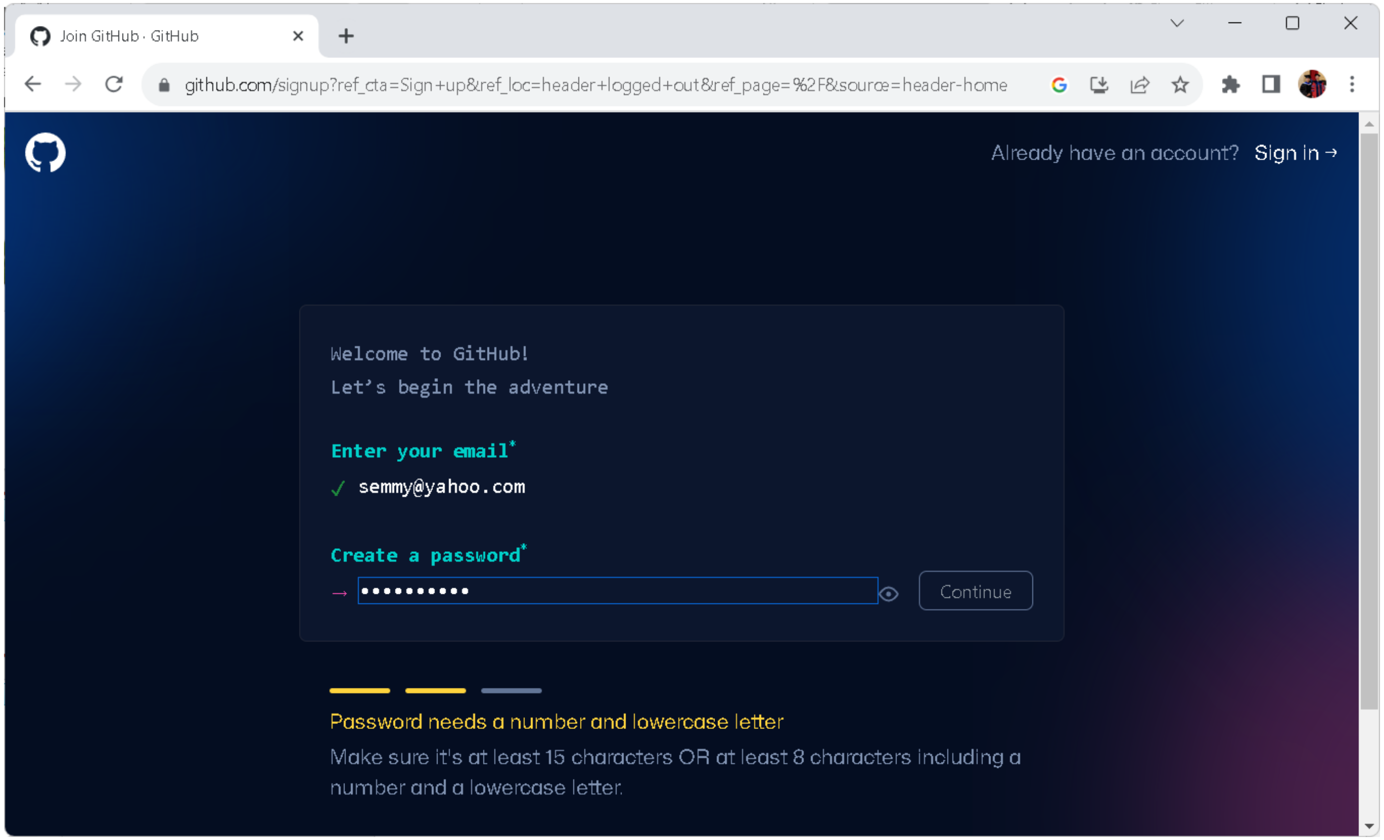


Silahkan click button **"Sign Up"** untuk memulai register new account.

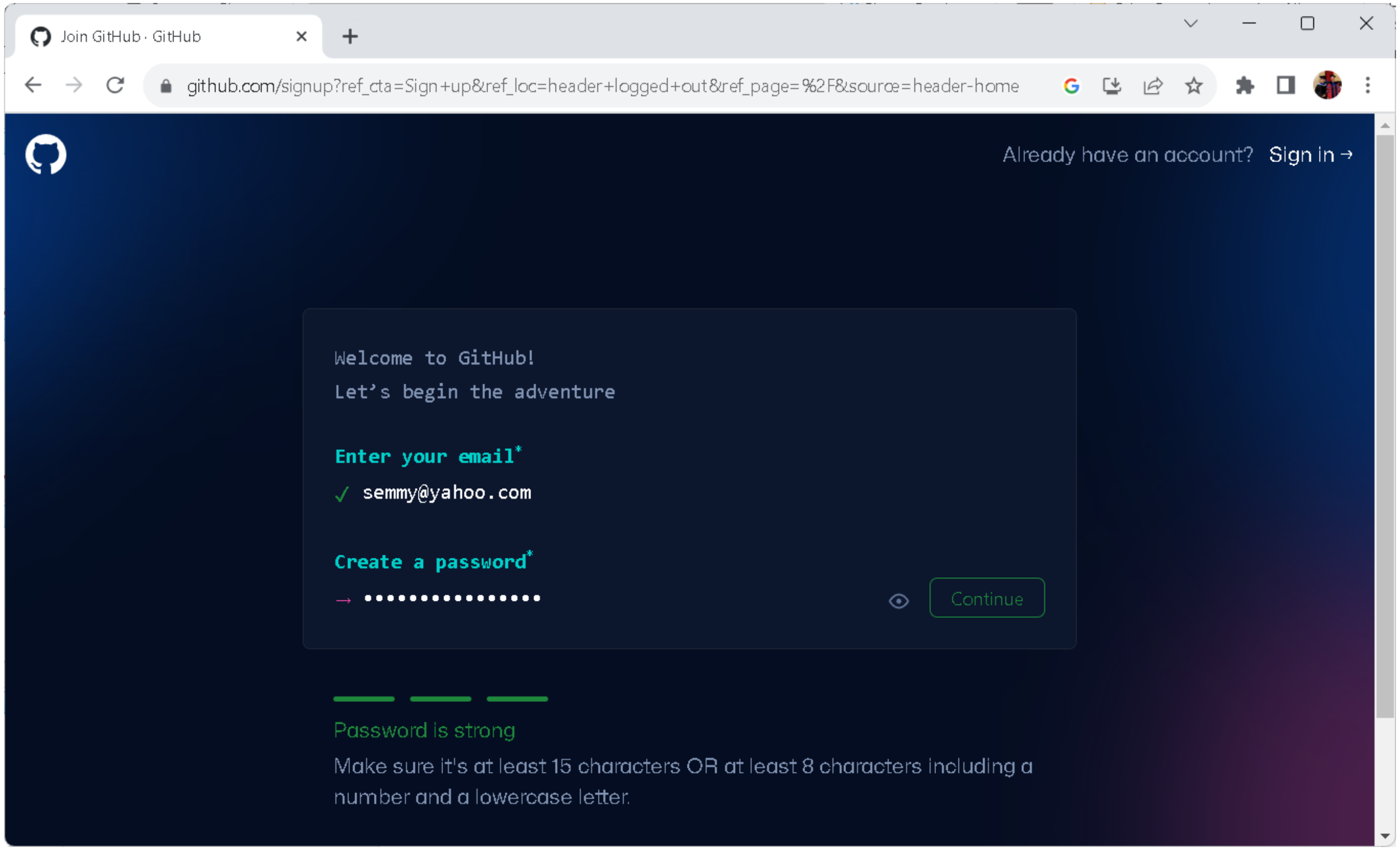
Create New Github Account



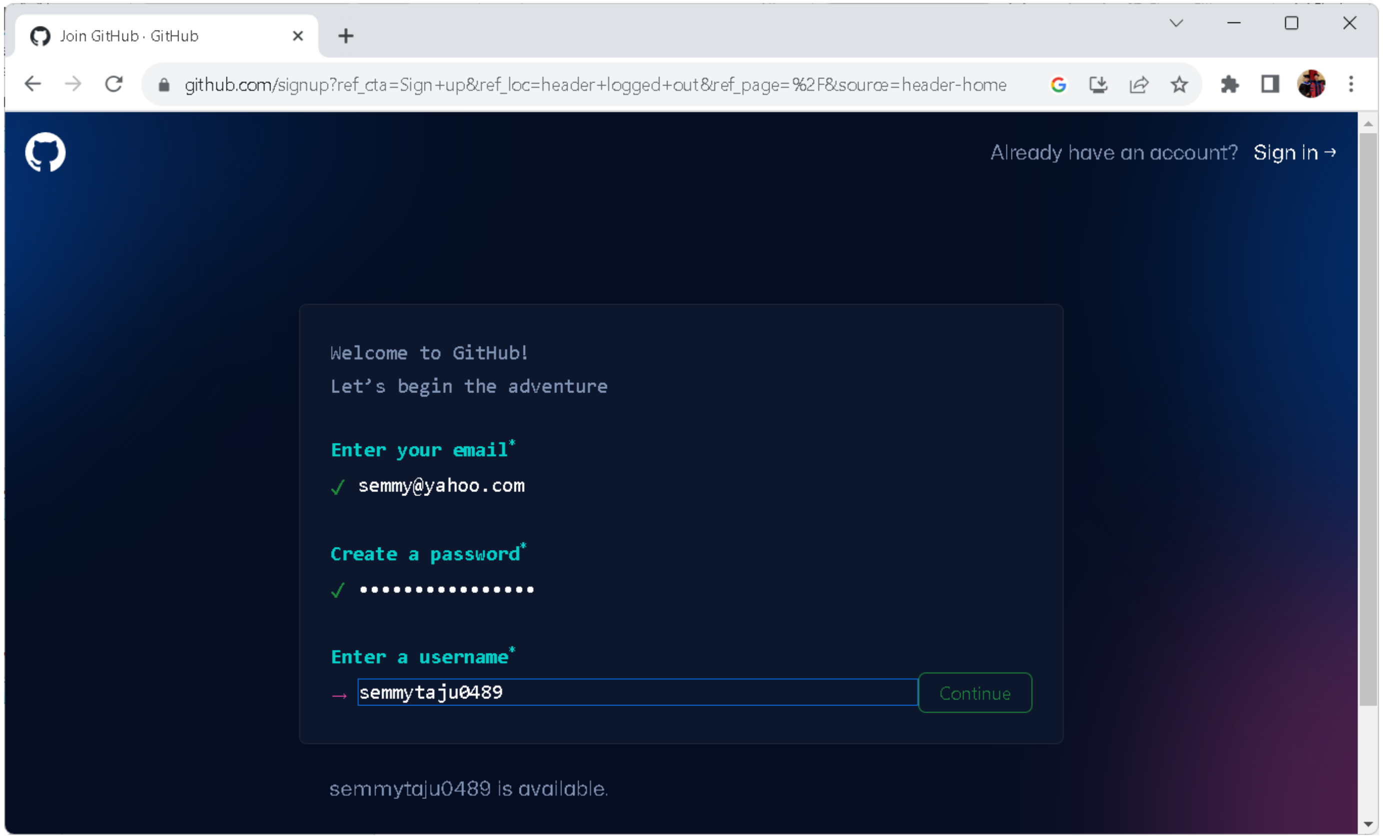
Create New Github Account



Create Strong Password

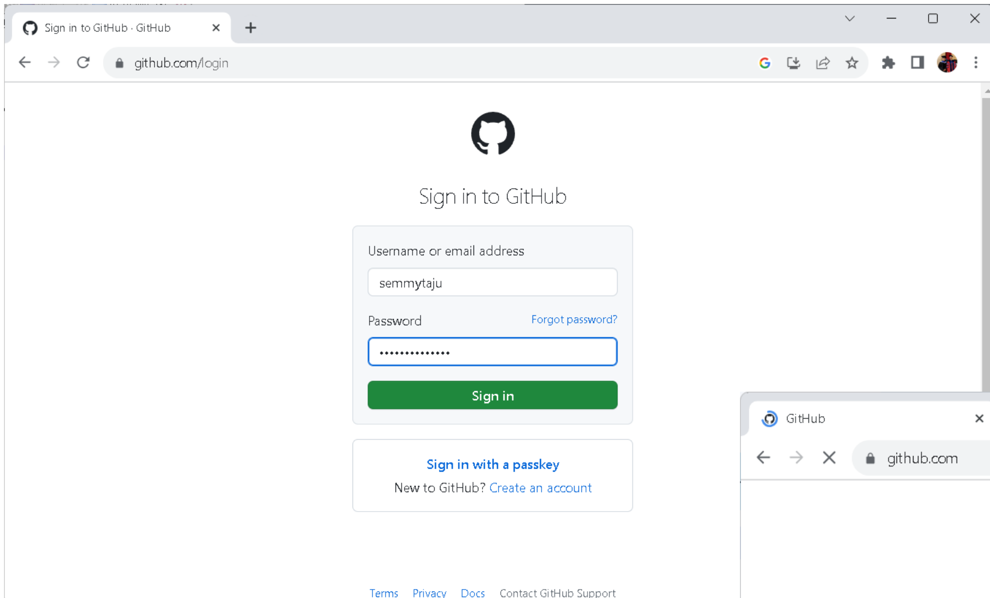


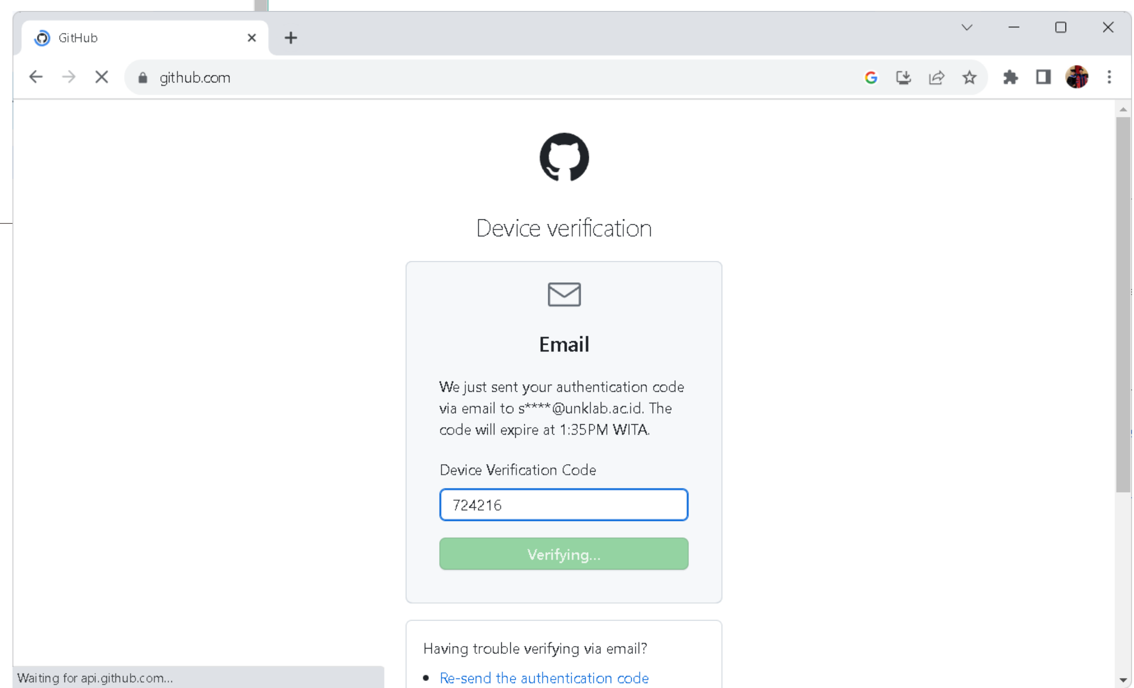
Create UserName



“Sign In” to Github

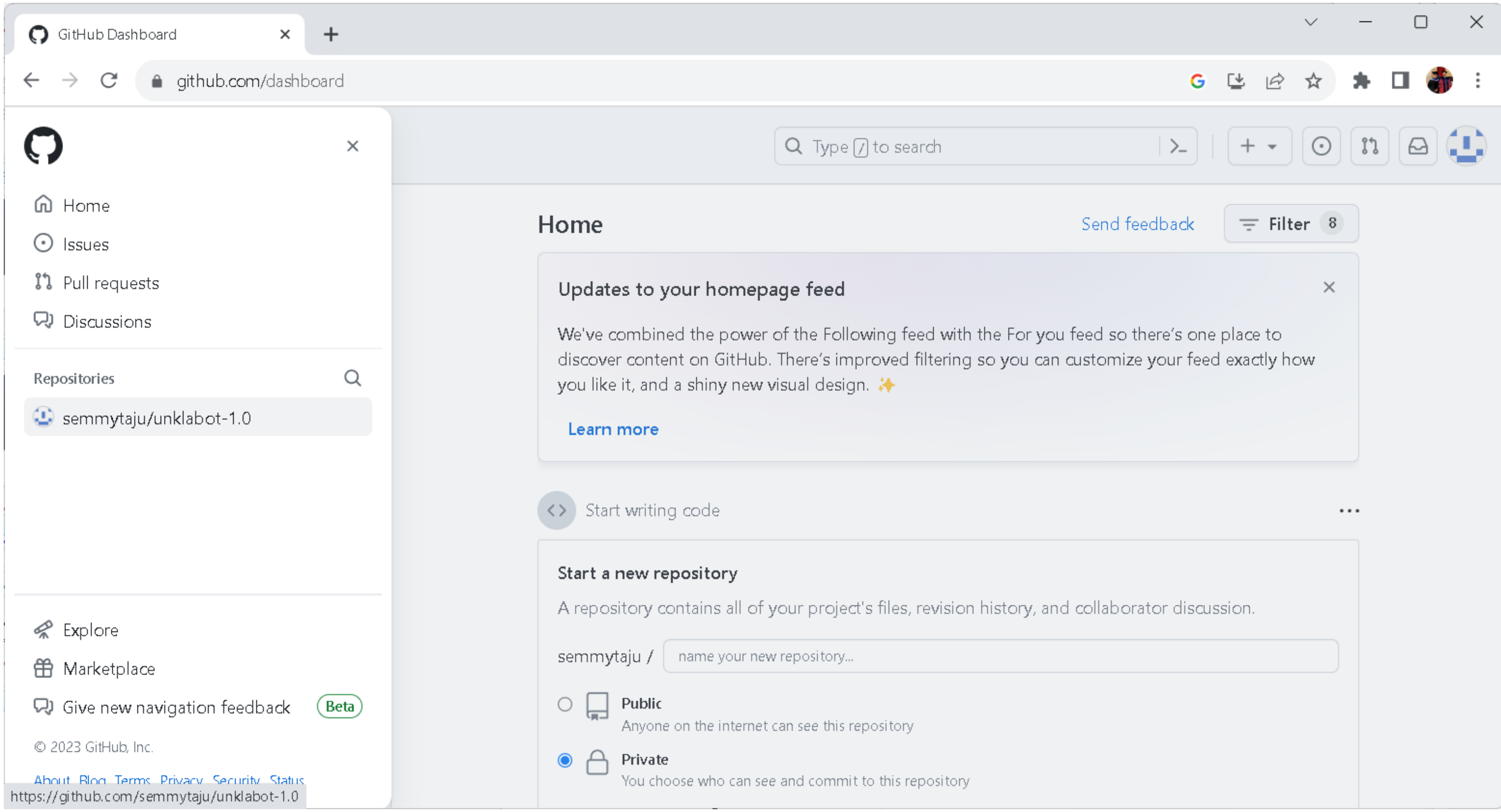
Github Login: https://github.com/login





Verifikasi code akan di kirim ke email.

Github Dashboard

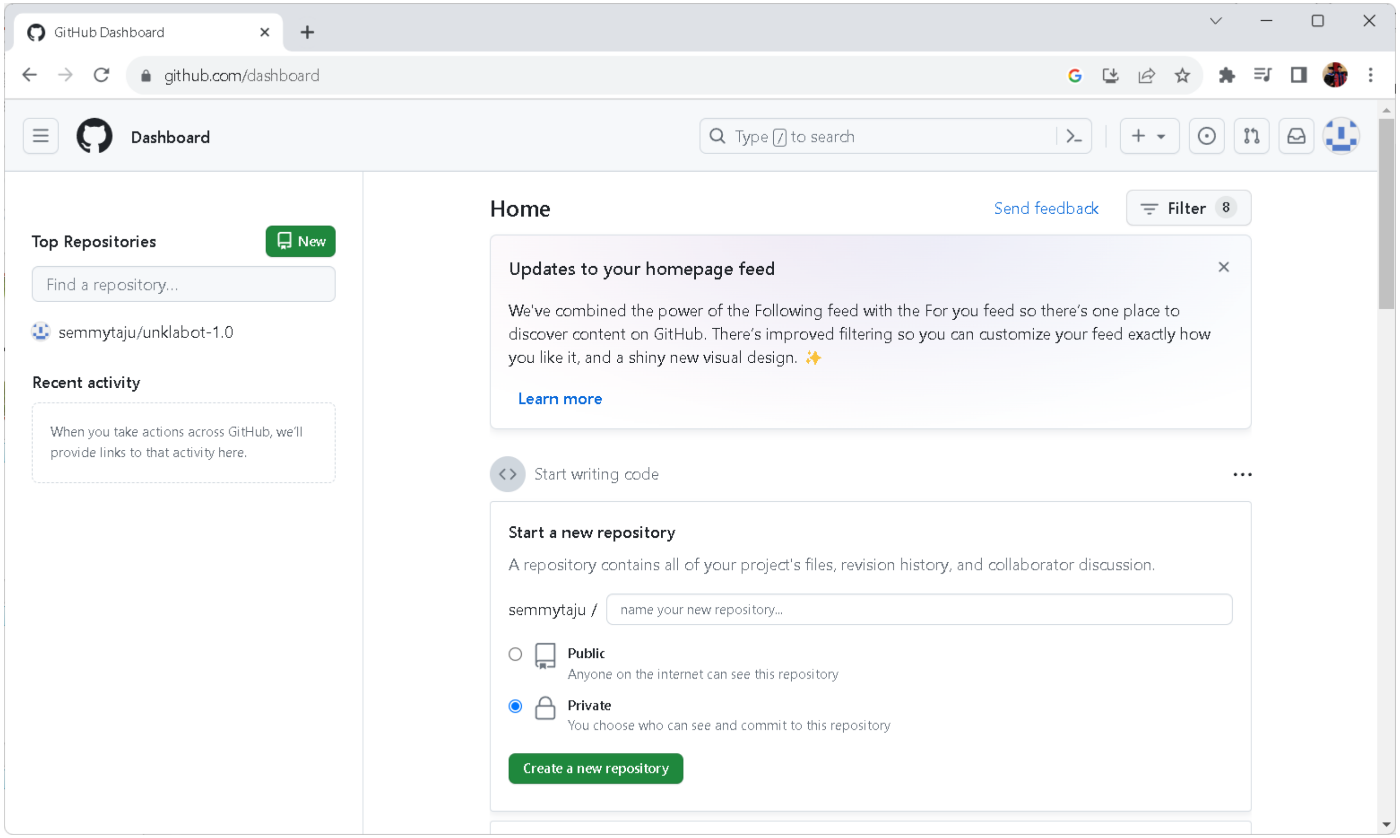


5 – Push Folders & Files to Github

(*git push = mengirimkan perubahan file*)

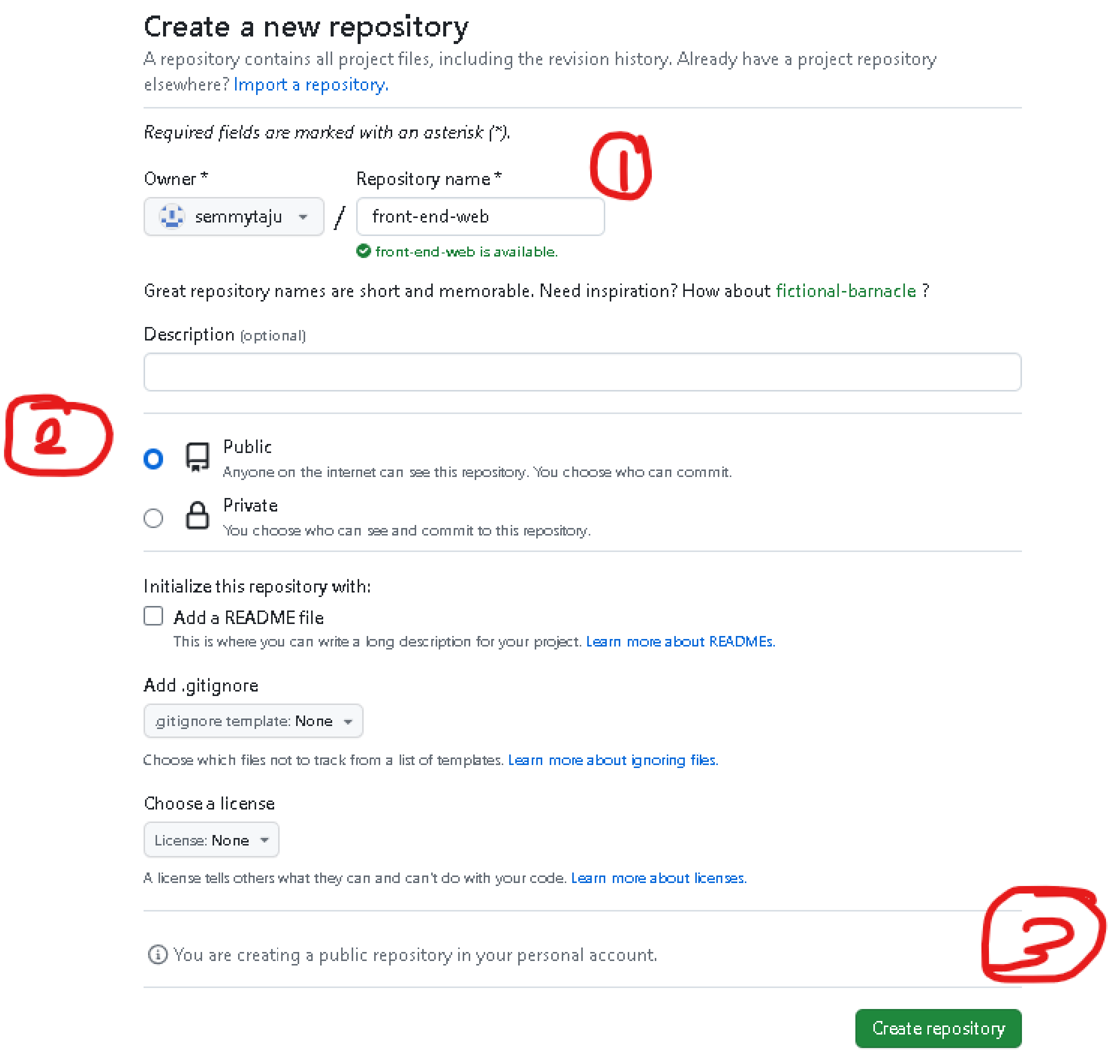
Create Repository Github

Kemudian buat repository dengan mengklik tanda + dan pilih **new repository**.

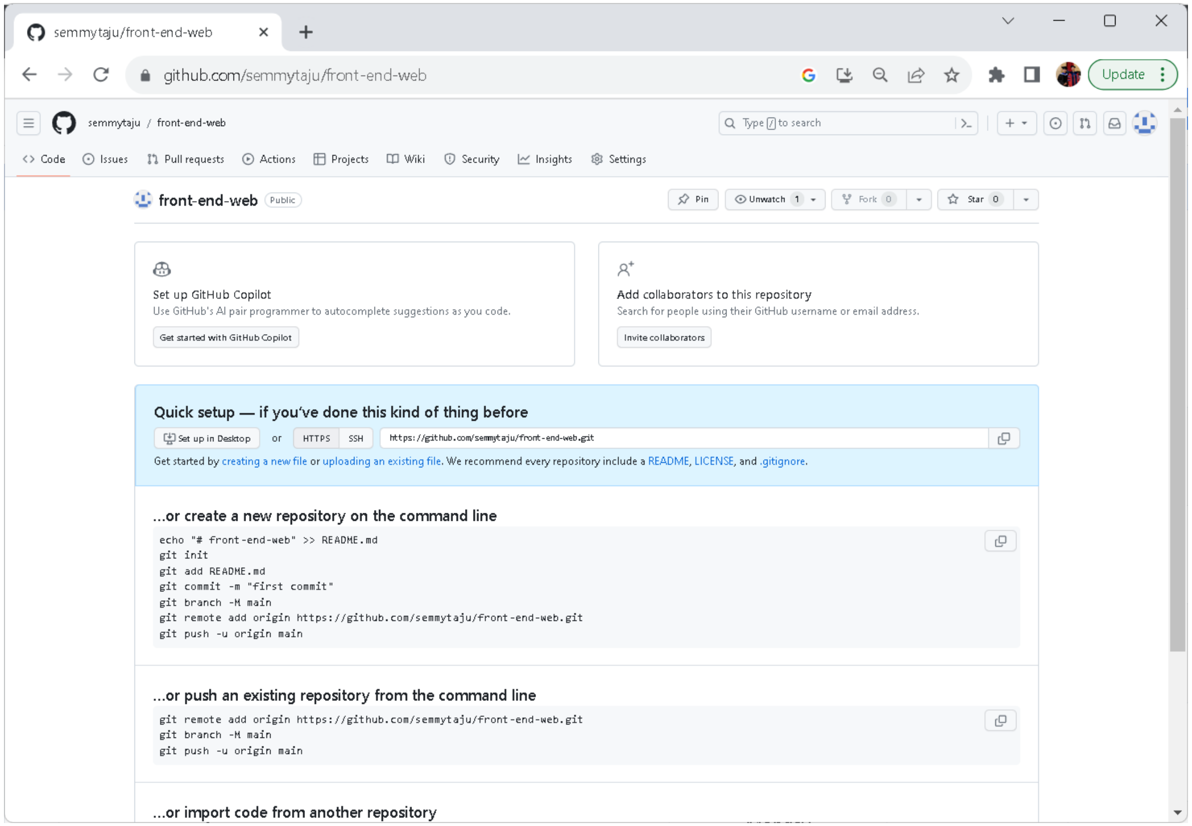


Public Repository Github

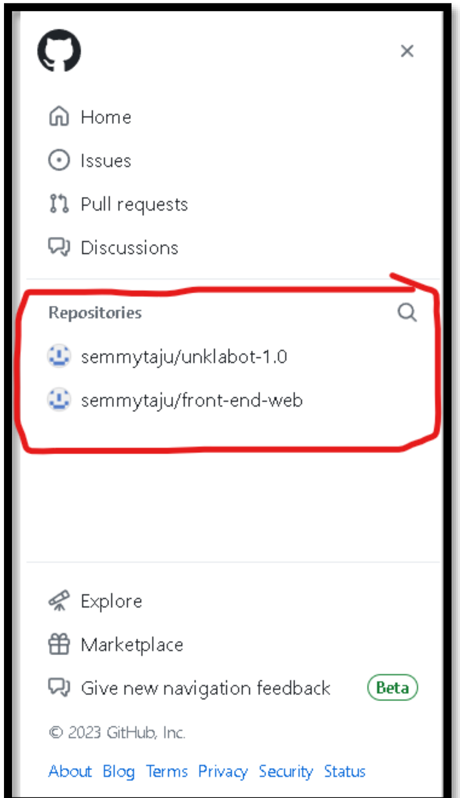
Next, masukan nama **repository** cloud/online yang akan dijadikan sebagai tempat code.

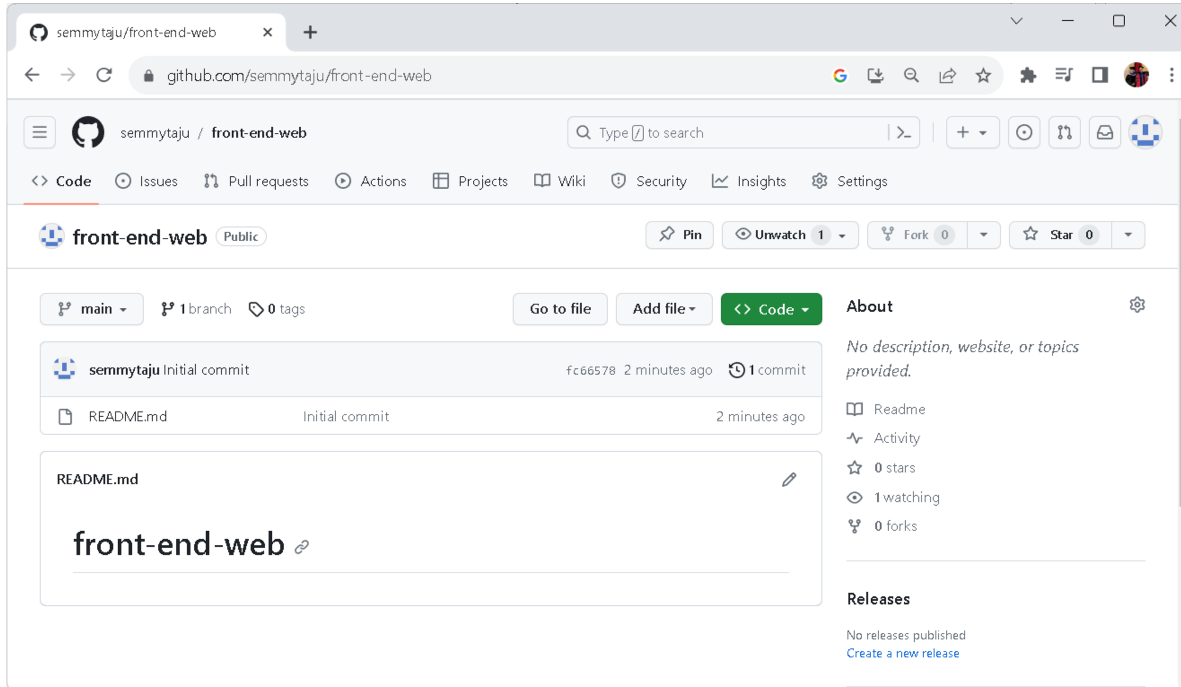


Quick Github Setup



Repository Created





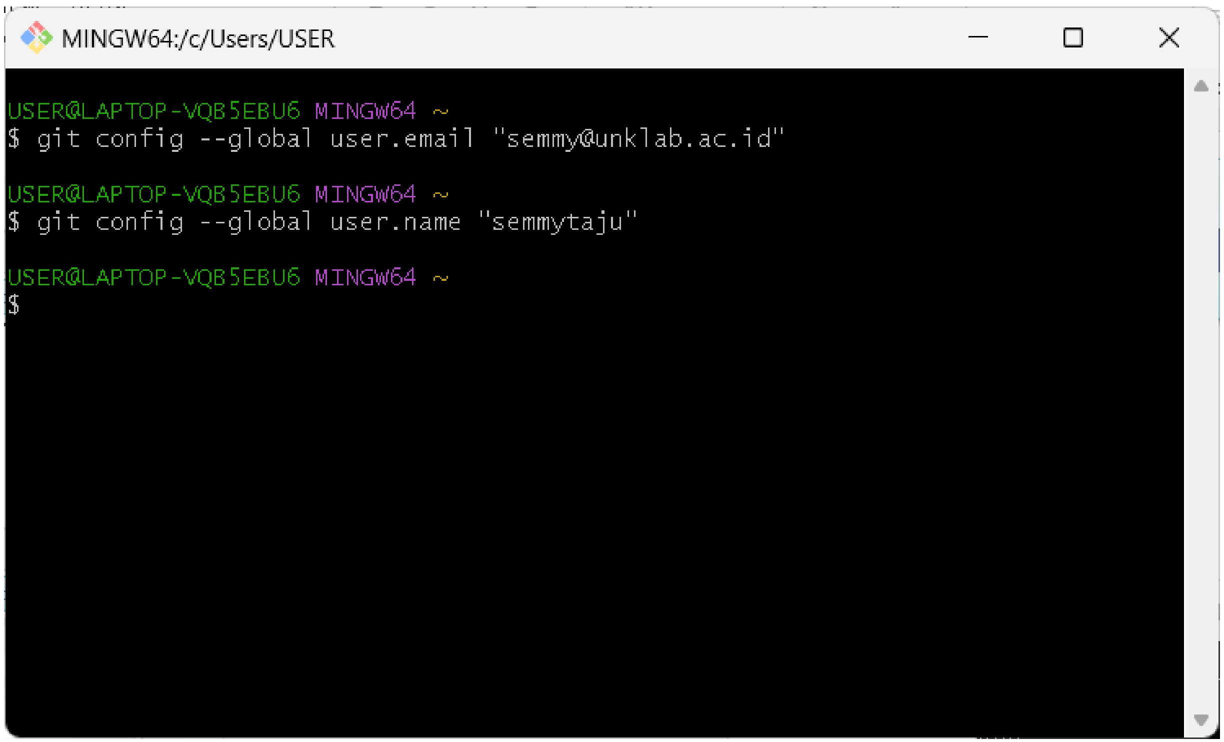
Setting Author Identity !!!

Author identity unknown (Please tell me who you are?). jika baru pertama install git maka kita konfigurasi kan email dan username git di lokal komputer kita untuk footprint atau jejak kita.

Run Following Script:

- git config --global user.email "semmy@unklab.ac.id"
- git config --global user.name "semmytaju"

Sesuaikan dengan kalian punya **email** dan **username**.



Example Directories

Berikut ini adalah beberapa contoh directory yang akan diupdate ke Github directory.

Folder Location: D:/test/front-end/

Open Git Bash:

Select/choose local drive:

- cd d:

Select/choose local folder:

- cd test

Select/choose local folder:

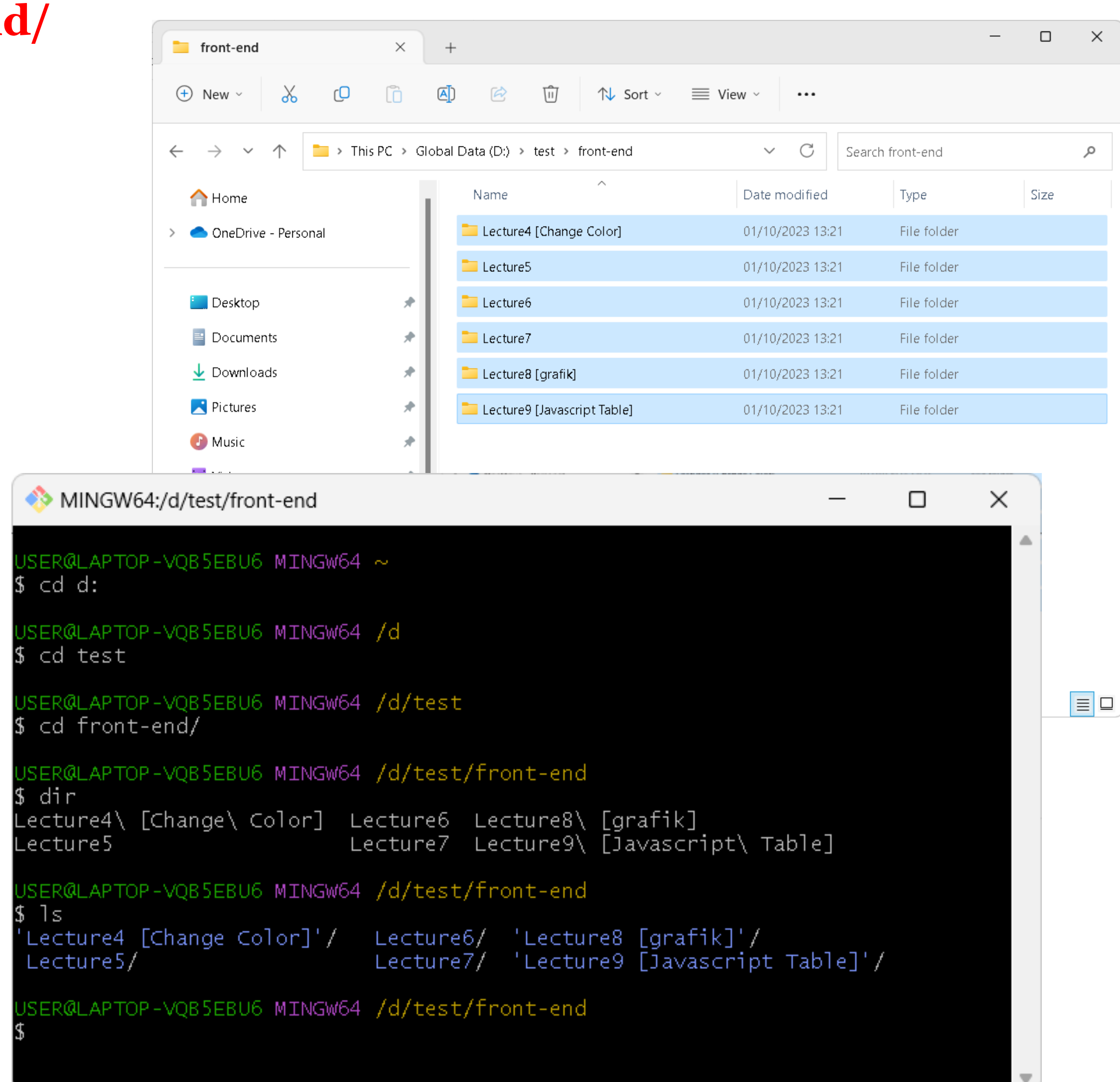
- cd front-end

View directory:

- dir

List directory:

- ls



"Sesuaikan dengan lokasi dan nama folder kalian."

Understand Git Commands

1) **git init**: Perintah ini digunakan untuk menginisialisasi repository Git baru. Ini membuat direktori kosong menjadi repositori Git yang siap untuk digunakan.

2) **git add .**: Perintah ini digunakan untuk menambahkan semua perubahan yang ada dalam direktori kerja Anda ke staging area. Dalam kasus ini, tanda "." mengacu pada semua file dan folder yang ada di direktori saat ini.

3) **git commit -m "commit pertama saya"**: Setelah perubahan ditambahkan ke staging area, kita perlu melakukan commit untuk menyimpan perubahan tersebut ke dalam sejarah repositori. Dalam perintah ini, kita memberikan pesan commit sebagai "*commit pertama saya*" yang digunakan untuk memberikan deskripsi singkat tentang perubahan yang dilakukan.

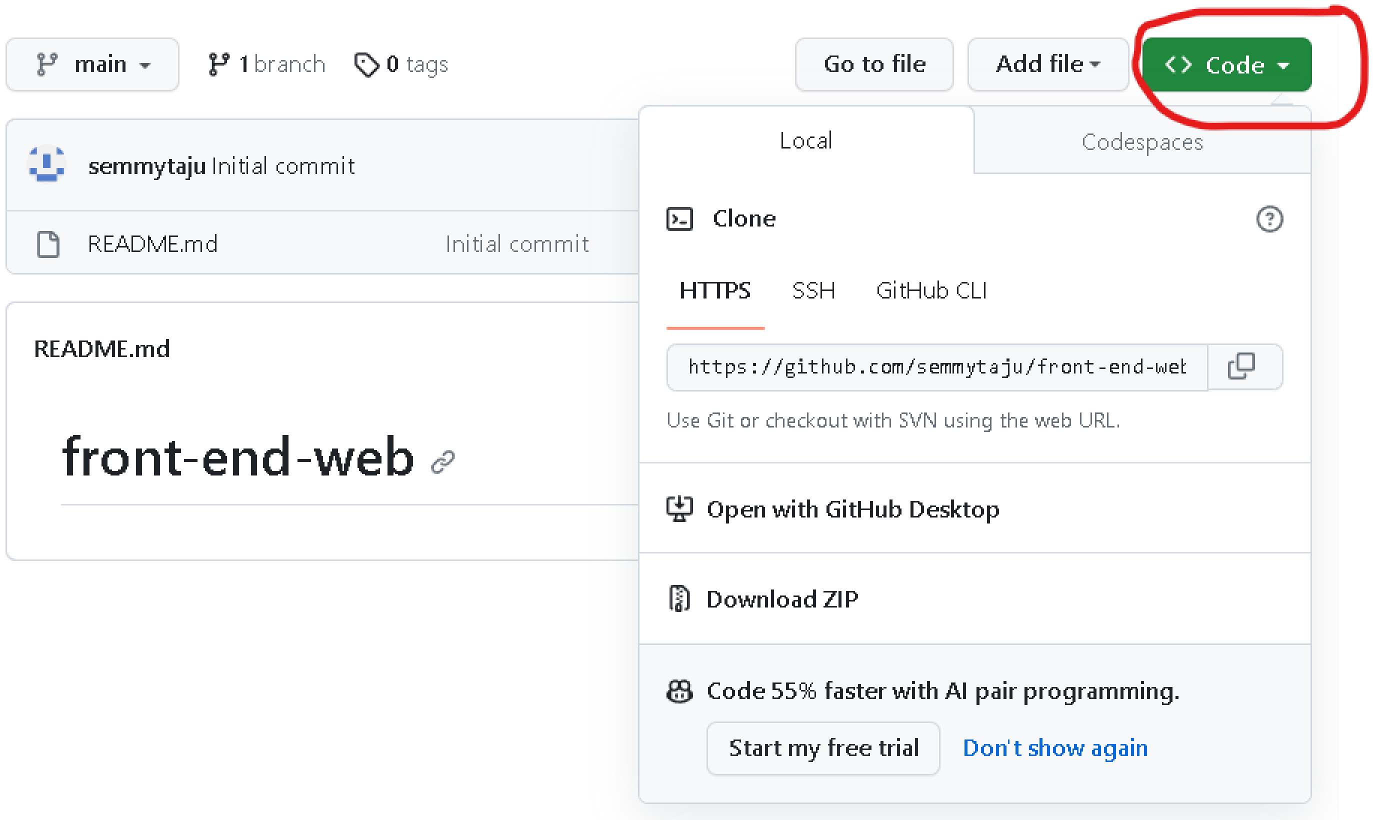
4) **git branch -m main**: Perintah ini digunakan untuk mengganti nama branch saat ini ke "main". Sebelumnya, **branch default dalam repositori Git adalah "master",** tetapi banyak repositori telah beralih ke menggunakan "main" sebagai nama branch default.

5) **git remote add origin https://github.com/semmytaju/front-end-web.git**: Perintah ini digunakan untuk menambahkan remote repository dengan nama "origin" yang terhubung ke URL yang diberikan. Ini adalah langkah yang diperlukan jika kita ingin menghubungkan repositori lokal dengan repositori di GitHub atau server Git lainnya.

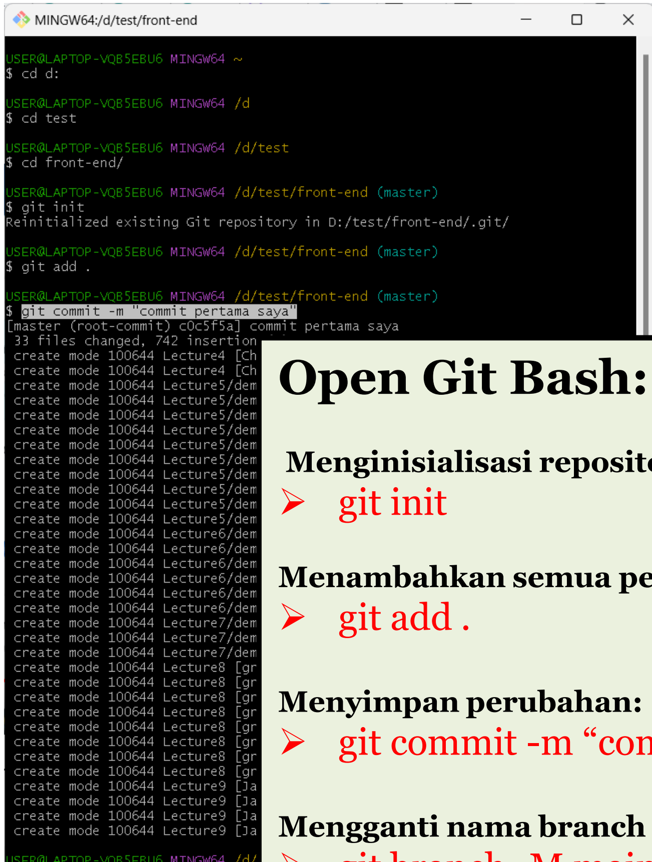
6) **git push -u origin main**: Perintah ini digunakan untuk mengirimkan perubahan yang sudah kita commit ke remote repository (GitHub dalam kasus ini). "-u" digunakan untuk mengatur "origin/main" sebagai branch default untuk melakukan push di masa depan. Artinya, kita bisa melakukan "git push" tanpa perlu menyebutkan branch lagi setiap kali.

Type Access To Github

Untuk menghubungkan **repositori lokal** dengan repositori di GitHub atau server Git maka kita perlu tipe akses yang sesuai. Disini saya memilih cara yang paling mudah yaitu dengan URL HTTPS.



Git Commit (Menyimpan Perubahan)



Gunakan perintah di bawah ini untuk menginisialisasi, menambahkan perubahan, melakukan commit, mengganti nama branch, menambahkan remote repository, dan mengirim perubahan ke GitHub menggunakan serangkaian perintah Git berikut.

Open Git Bash:

Menginisialisasi repository Git baru:

- git init

Menambahkan semua perubahan yang ada dalam direktori:

- git add .

Menyimpan perubahan:

- git commit -m “commit pertama saya”

Mengganti nama branch saat ini ke "main":

- git branch -M main

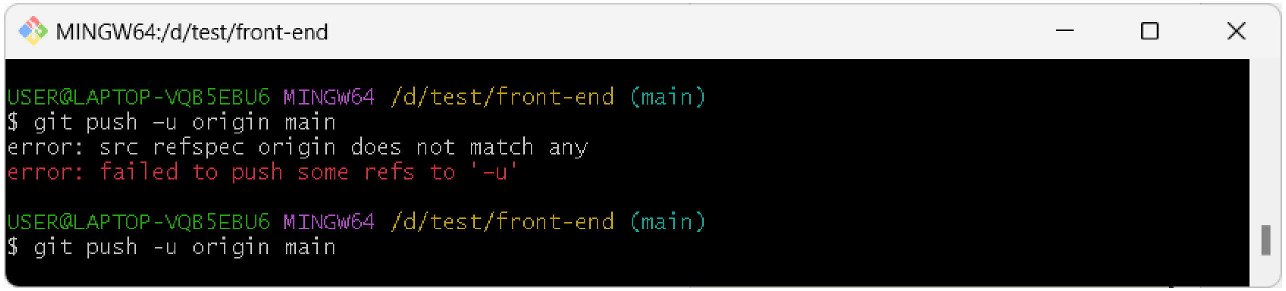
Menambahkan remote repository dengan nama "origin" yang terhubung ke URL:

- git remote add origin https://github.com/semmytaju/front-end-web.git

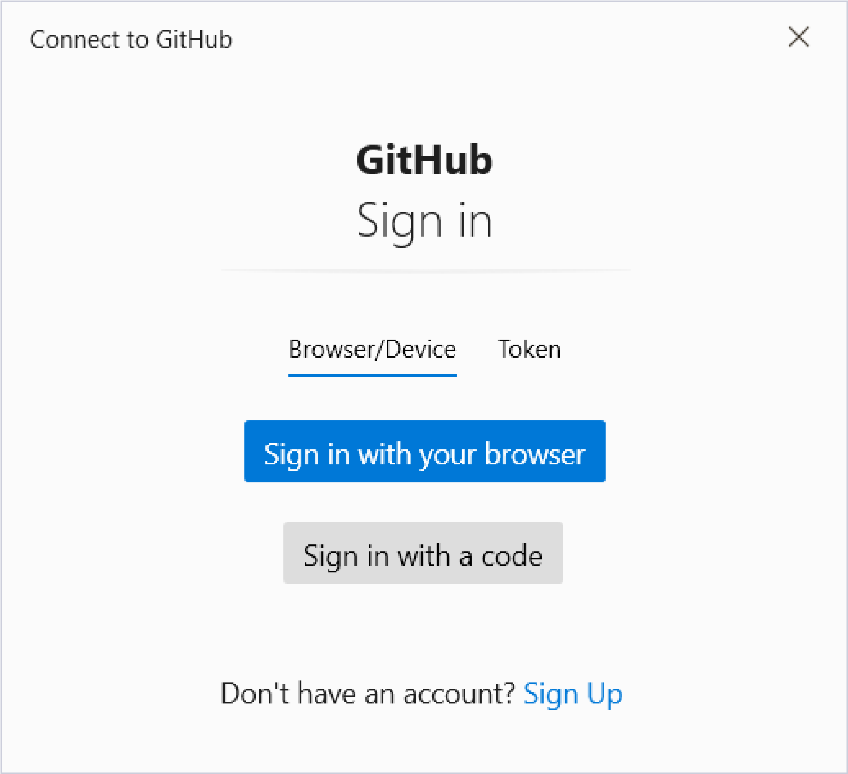
Mengirimkan perubahan yang sudah di commit ke remote repository github:

- git push -u origin main

Get Push (kirim perubahan)



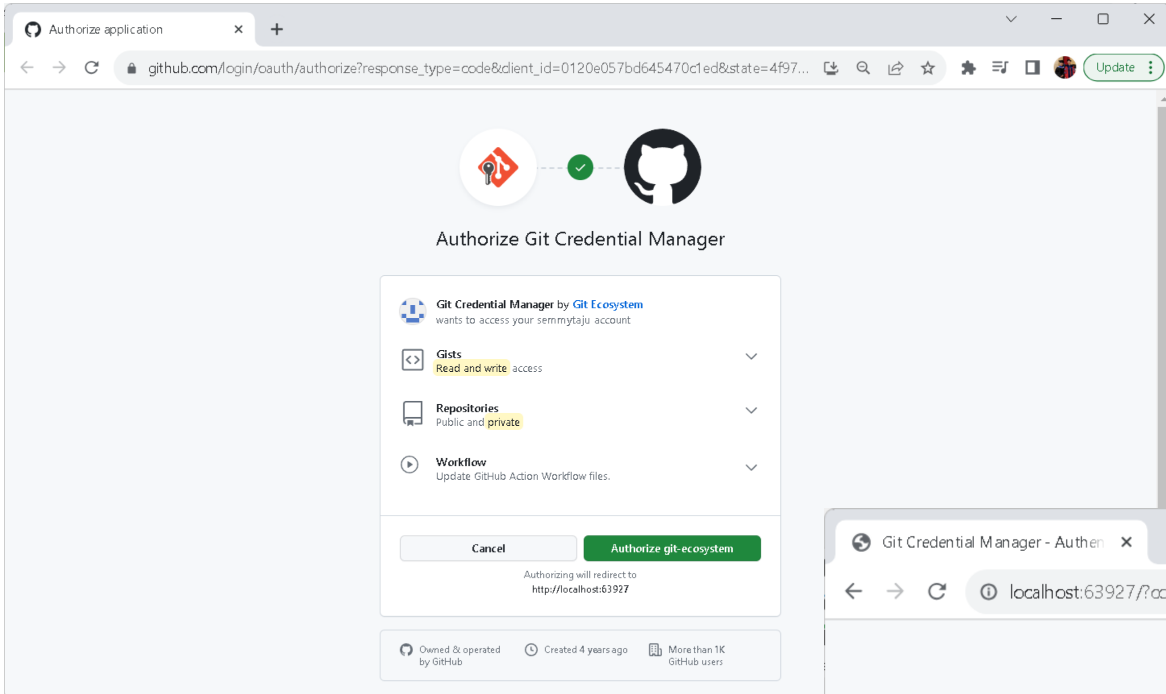
Form login ke Github akan muncul secara otomatis.

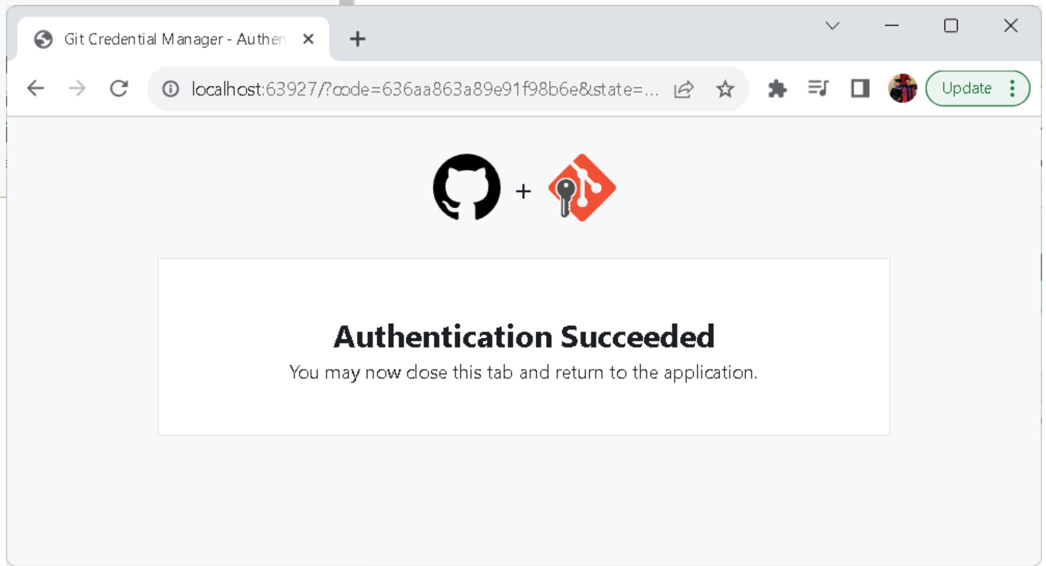


Pilih "*Sign In with you browser*".

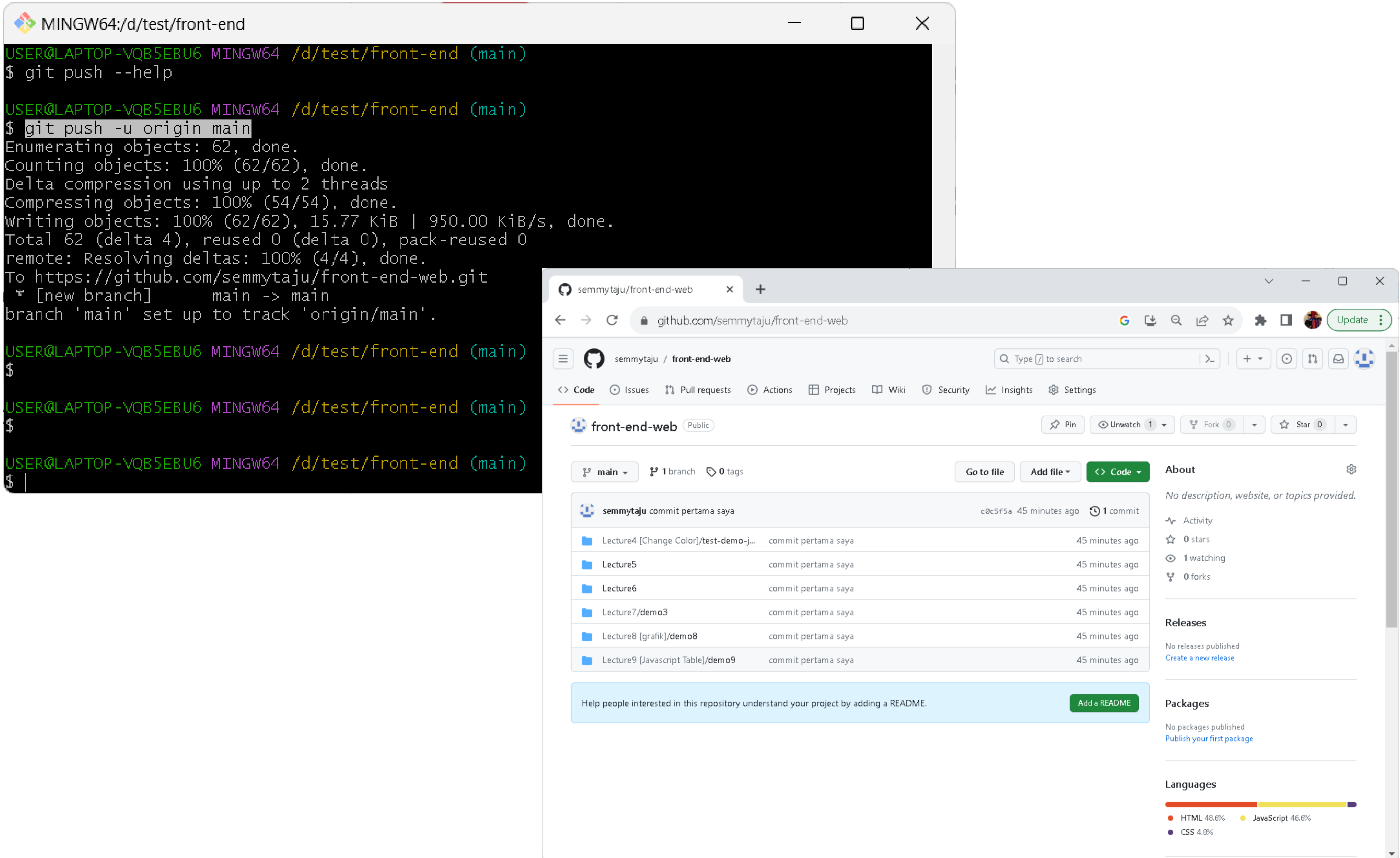
Authorize Git Credential Manager

Pilih “Sign In with you browser”.

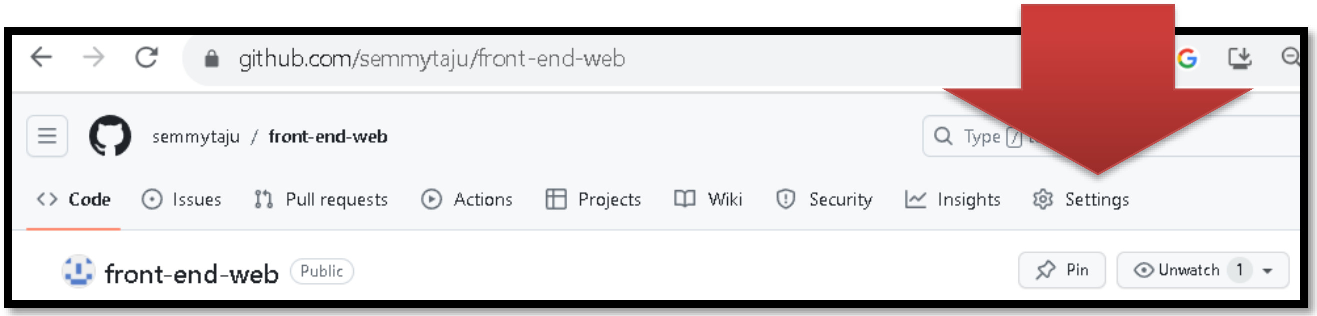


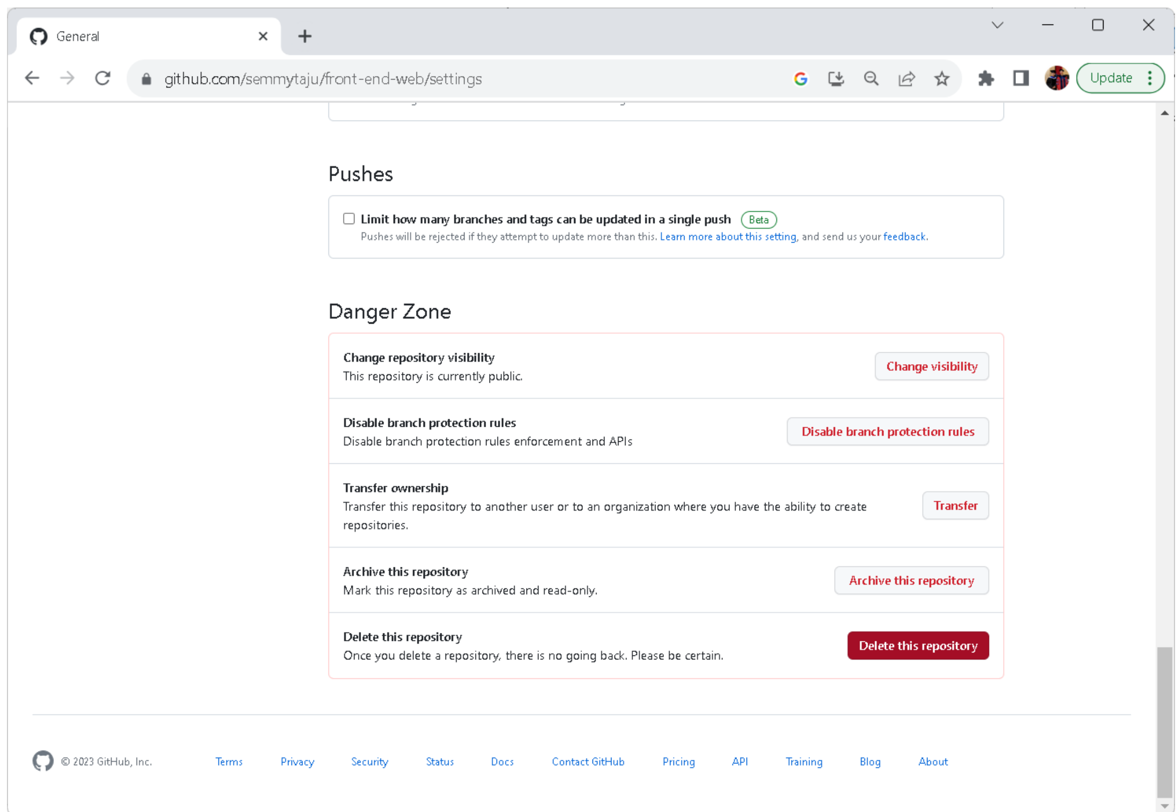


Get Push Successfully



Privately Share & Delete Repository



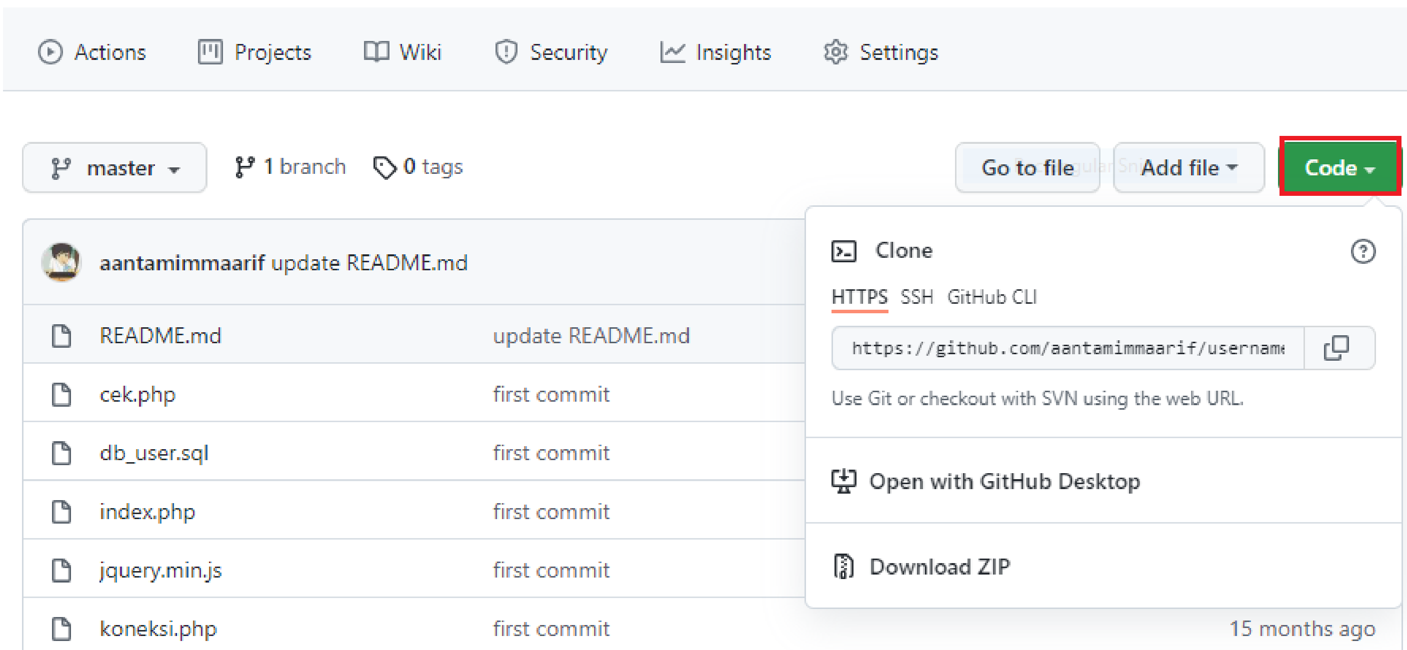


6 – Clone Repository GitHub

(*git clone = membuat salinan/copy ke repository local*)

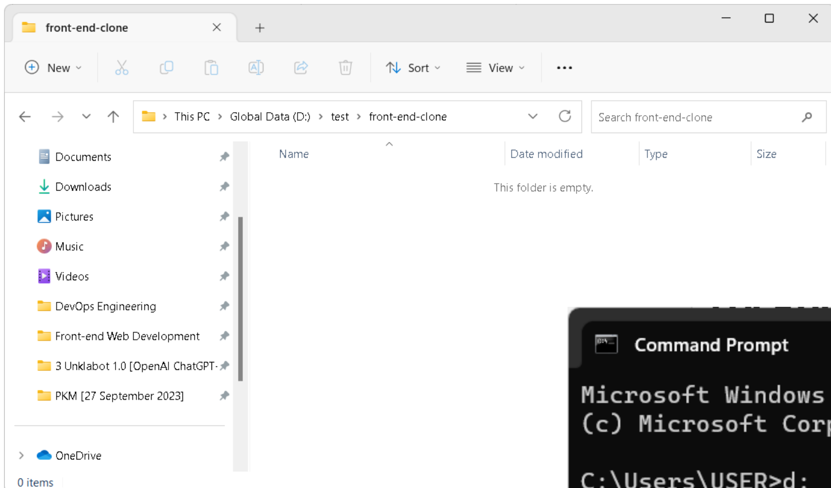
Clone Repositori GitHub

Kita akan mencoba melakukan **CLONE** sebuah repositori di GitHub, click tombol berwarna hijau “Code” dan pilih tab **HTTPS** kemudian salin URL yang berakhiran .git tersebut.

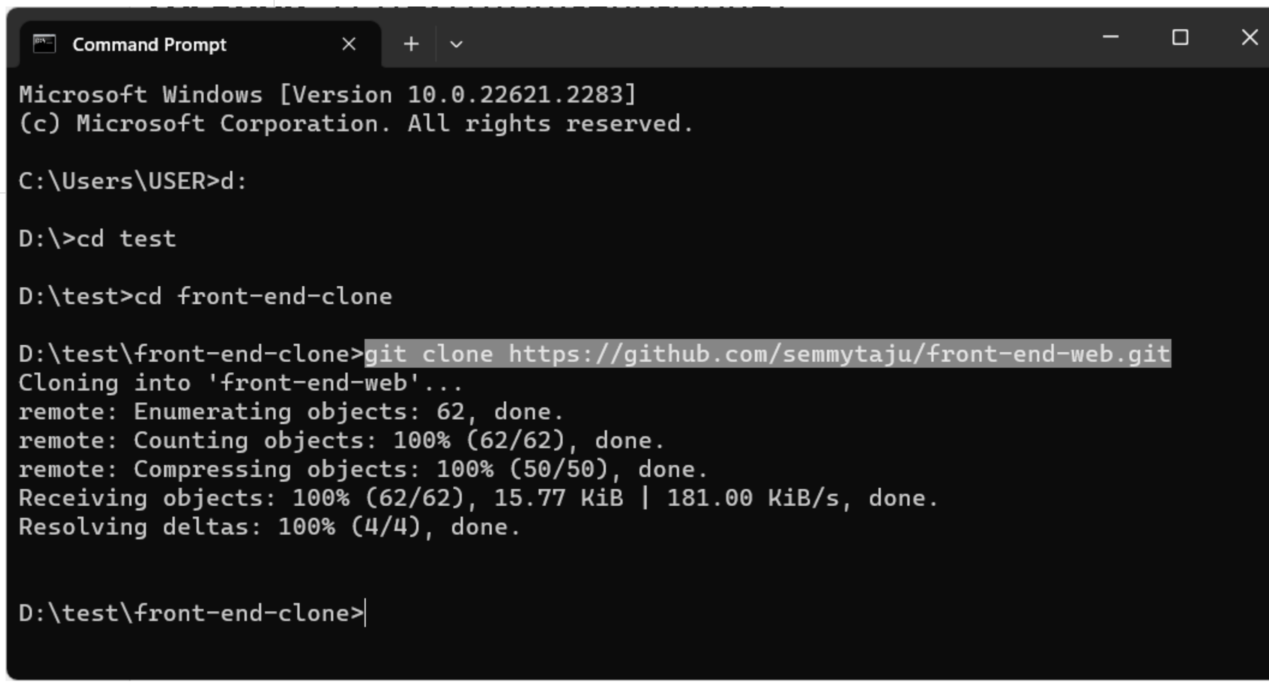


Create New Local Folder

Location: D:/test/front-end-clone/



“Sesuaikan dengan lokasi dan nama folder kalian.”



Open CMD Windows:

Select/choose local drive:

- d:

Select/choose local folder:

- cd test

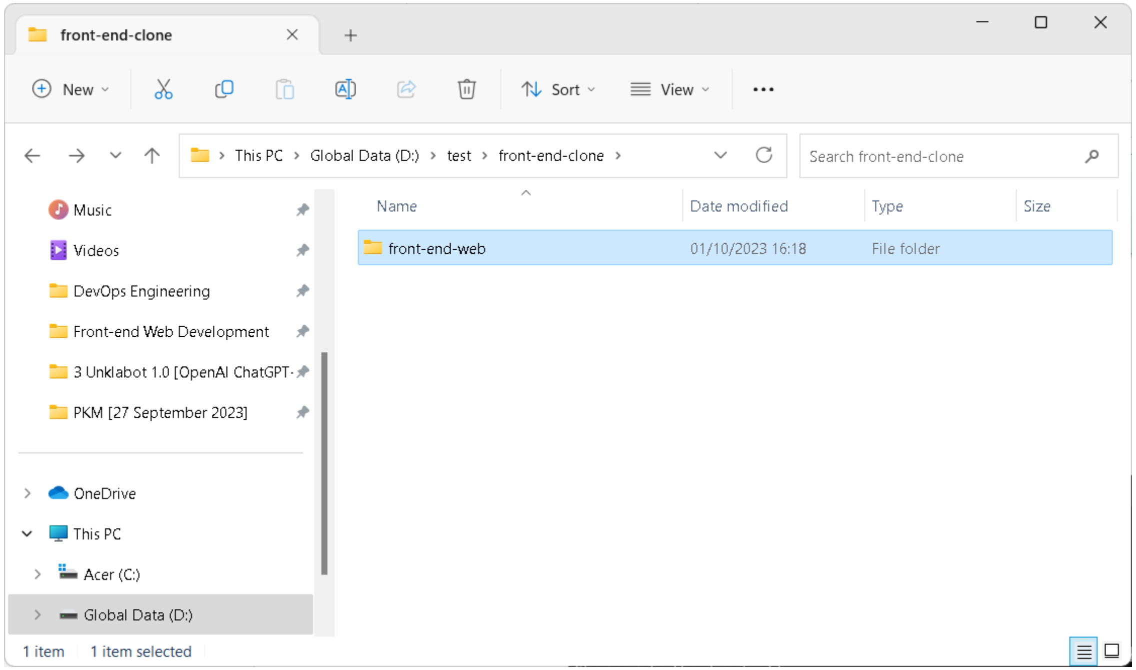
Select/choose local folder:

- cd front-end-clone

Clone github directory:

- git clone https://github.com/semmytaju/front-end-web.git

Repository Github Copied



Clone Repository menggunakan SSH Github:
https://aantamim.id/git-windows-install/

Cara Berkontribusi di dalam Project Open Source Github:
https://www.petanikode.com/github-workflow/

Cara Install Git & Git Bash di Windows:

https://youtu.be/IigslfdR7vI

Cara Upload Folder & File ke Github:

https://youtu.be/WRsK4f9fmY8

7 – Push ke GitHub dari Ubuntu 22.04

Install & Setting Identitas Git

Menginstal Git di Ubuntu 22.04 Server:

- sudo apt install git
- git --version

Setting Identitas Git:

- git config --global user.name "Nama Anda"
- git config --global user.email "alamat@email.com"

Push ke GitHub:

- mkdir testdirdevops
- git add .
- git commit -m "Pesan commit Anda"
- git push origin master

Clone dari GitHub ke Ubuntu 22.04 Server:

- git clone <URL Repositori>

Exercise *for* Students

Class Activities

1) Students diharapkan untuk bisa menginstall Git di ubuntu 22.04.

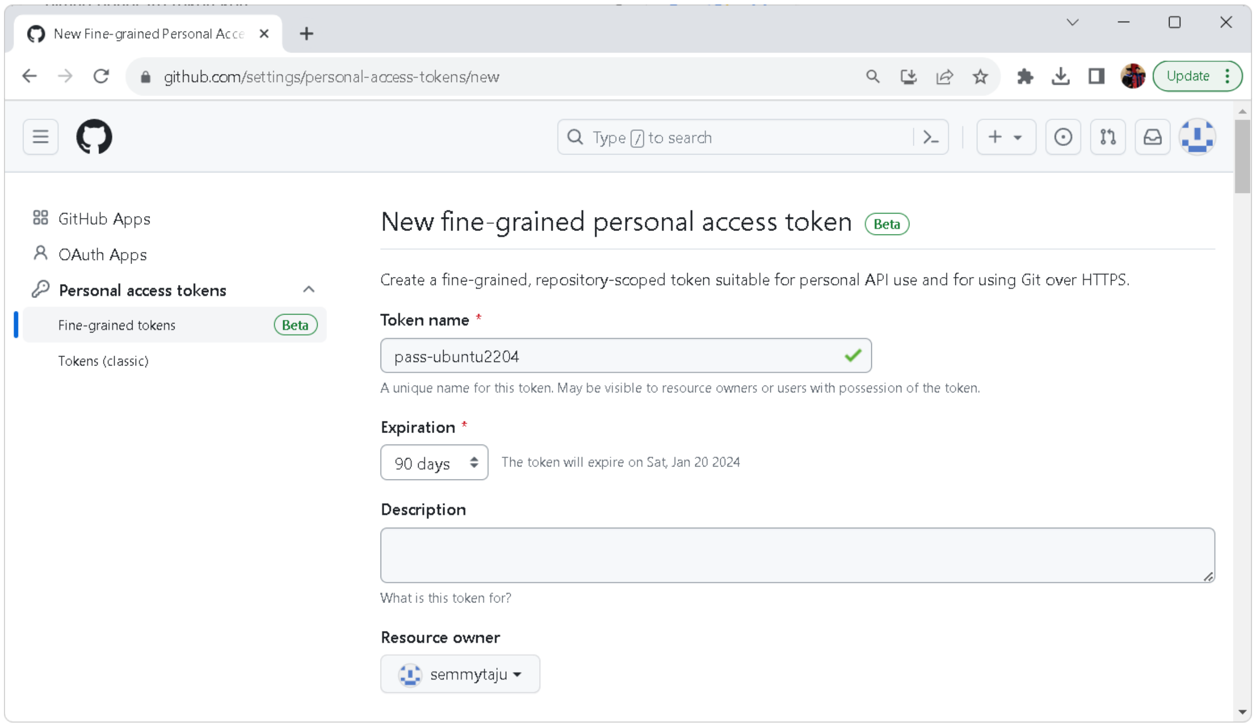
2) Students diharapkan bisa membuat new account di Github.

3) Students bisa melakukan **PUSH** semua *folder exercises* yang dibuat dari Ubuntu 22.04 ke Github.

4) Students bisa melakukan **CLONE** repository di github ke *local folder* di Ubuntu 22.04.

Personal Access Tokens (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a **password for Git over HTTPS**, or can be used to authenticate to the API over Basic Authentication.



Creating a personal access token:

1) In the upper-right corner of any page, click your profile photo, then click Settings.
2) In the left sidebar, click Developer settings.
3) In the left sidebar, click Personal access tokens.
4) Click Generate new token.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T